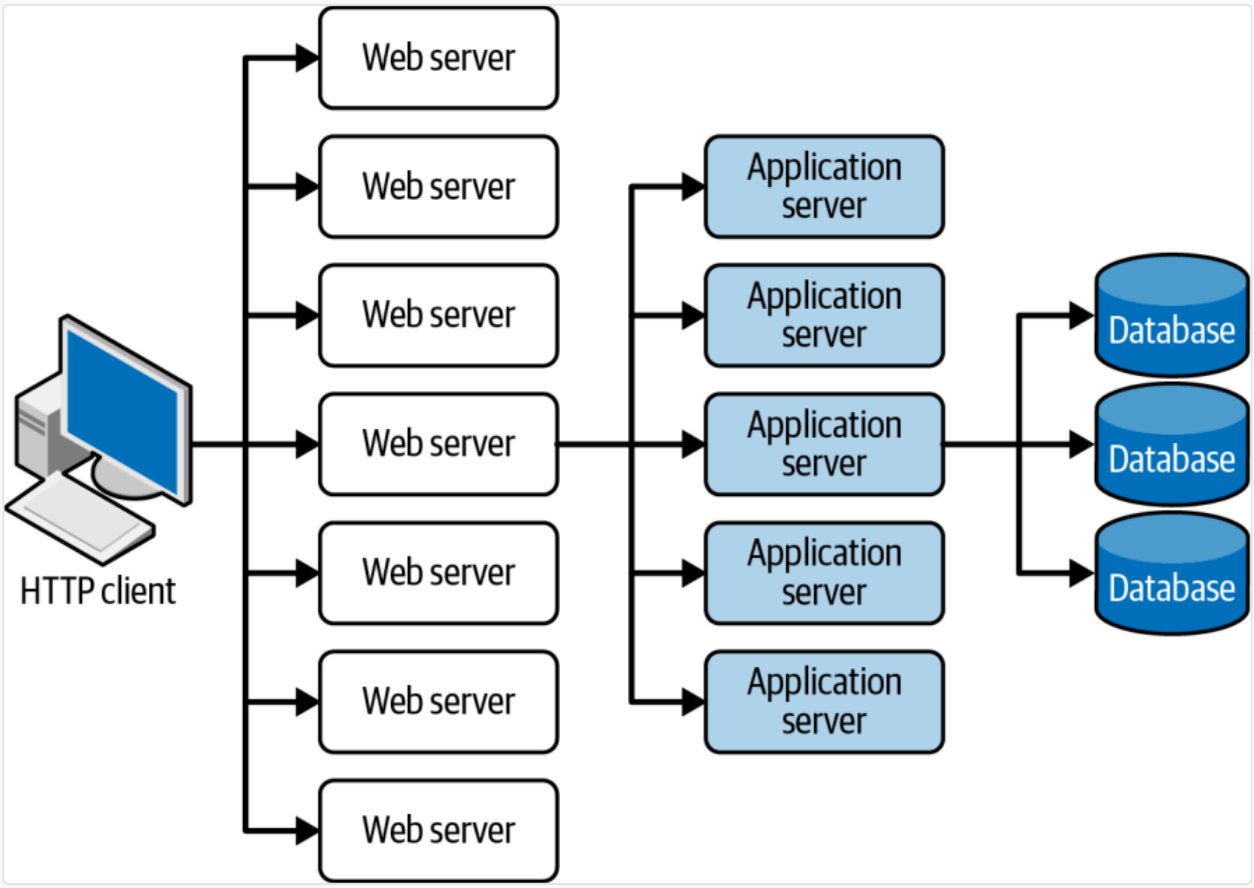
فصل ۱۶: سبک معماری مبتنی بر فضا

بیشتر برنامه‌های کسب‌وکاریِ مبتنی بر وب از جریان درخواست (request flow) کلیِ یکسانی پیروی می‌کنند: درخواستی از مرورگر نخست به سرور وب (web server) می‌رسد، سپس به سرور برنامهٔ کاربردی و سرانجام به سرور پایگاه‌داده (database server). هرچند این جریان درخواستِ متداول برای شمار اندکی از کاربران هم‌زمان کار می‌کند، با افزایش بار کاربران هم‌زمان گلوگاه‌ها پدیدار می‌شوند: نخست در لایهٔ سرور وب، سپس در لایهٔ سرور برنامهٔ کاربردی و سرانجام در لایهٔ سرور پایگاه‌داده.

واکنش معمول هنگامی که افزایش بار کاربران گلوگاه پدید می‌آورد، مقیاس‌دهیِ افقی (scale out) سرورهای وب است. این کار نسبتاً آسان و کم‌هزینه است و گاهی هم جواب می‌دهد. اما در بیشتر موارد بارِ بالای کاربران، مقیاس‌دهیِ افقیِ لایهٔ سرور وب تنها گلوگاه را به سرور برنامهٔ کاربردی منتقل می‌کند. مقیاس‌دهیِ سرورهای برنامهٔ کاربردی ممکن است پیچیده‌تر و پرهزینه‌تر از مقیاس‌دهیِ سرورهای وب باشد و معمولاً تنها گلوگاه را به سرور پایگاه‌داده منتقل می‌کند که مقیاس‌دهی‌اش از آن هم دشوارتر و پرهزینه‌تر است. حتی اگر بتوانید پایگاه‌داده را مقیاس‌پذیر کنید، سرانجام به یک توپولوژیِ مثلثی‌شکل می‌رسید که پهن‌ترین بخش آن سرورهای وب (آسان‌ترین بخش برای مقیاس‌دهی) و باریک‌ترین بخش آن پایگاه‌داده (دشوارترین بخش برای مقیاس‌دهی) است؛ چنان‌که در شکل ۱۶-۱ نشان داده شده است.

در هر برنامهٔ پرحجمی (high-volume application) که بار کاربرانِ هم‌زمانِ زیادی دارد، پایگاه‌داده معمولاً عامل محدودکنندهٔ نهایی در این است که برنامه چند تراکنش را می‌تواند به‌طور هم‌زمان پردازش کند. هرچند فناوری‌های گوناگون کش‌گذاری و محصولات مقیاس‌دهیِ پایگاه‌داده می‌توانند کمک کنند، مقیاس‌دهیِ افقیِ یک برنامهٔ معمولی برای بارهای بسیار سنگین همچنان کاری بسیار دشوار است.



شکل ۱۶-۱. محدودیت‌های مقیاس‌پذیری در یک توپولوژیِ سنتیِ مبتنی بر وب

سبک معماری مبتنی بر فضا `(space-based architecture)` به‌طور ویژه برای رسیدگی به مسائلی طراحی شده است که با مقیاس‌پذیری، کشسانی و هم‌روندیِ بالا سروکار دارند. این سبک برای برنامه‌هایی که حجم کاربران هم‌زمانشان متغیر و پیش‌بینی‌ناپذیر است نیز سبک معماریِ سودمندی است. اغلب بهتر است مسئلهٔ مقیاس‌پذیریِ بسیار زیاد و متغیر را در سطح معماری حل کنیم، تا اینکه بکوشیم پایگاه‌داده را به‌صورت افقی مقیاس‌پذیر کنیم یا فناوری‌های کش‌گذاری را بعداً به معماری‌ای بیفزاییم که به آن خوبی مقیاس‌پذیر نیست.

توپولوژی

معماری مبتنی بر فضا نامش را از مفهوم فضای تاپل `(tuple space)` گرفته است؛ یعنی تکنیک استفاده از چندین پردازندهٔ موازی که از راه حافظهٔ مشترک `(shared memory)` با هم ارتباط برقرار می‌کنند. سیستم‌های مبتنی بر فضا با جایگزین‌کردنِ پایگاه‌دادهٔ مرکزی، که در سیستم نقش یک قید هم‌گام را دارد، با شبکه‌های دادهٔ درون‌حافظه‌ایِ تکثیرشده `(replicated in-memory data grids)`، به مقیاس‌پذیری، کشسانی و کاراییِ بالا دست می‌یابند.

داده‌های برنامه در حافظه نگه داشته می‌شوند و میان همهٔ واحدهای پردازشِ `(processing units)` فعال تکثیر می‌شوند. هنگامی که یک واحد پردازش داده‌ای را به‌روز می‌کند، آن داده را از راه یک پمپ داده `(data pump)`، معمولاً با پیام‌رسانی و صف‌های ماندگار `(persistent queues)`، به‌صورت ناهم‌گام به پایگاه‌داده می‌فرستد. واحدهای

پردازش با افزایش و کاهش بار کاربران به‌صورت پویا راه‌اندازی و خاموش می‌شوند و همین امر به مقیاس‌پذیریِ متغیر پاسخ می‌دهد. از آنجا که هیچ پایگاه‌دادهٔ مرکزی‌ای در پردازش تراکنشیِ استانداردِ برنامه دخالت ندارد، گلوگاه پایگاه‌داده از میان برداشته می‌شود. این امر مقیاس‌پذیریِ تقریباً بی‌پایانی را درون برنامه ممکن می‌سازد.

معماری مبتنی بر فضا سبک معماریِ پیچیده‌ای است که از مصنوعات گوناگونی تشکیل می‌شود که برای پردازش یک درخواست واحد با هم کار می‌کنند. مصنوعات اصلی آن عبارت‌اند از:

واحدهای پردازش `(processing units)`: کارکرد برنامه را در بر دارند.

میان‌افزار مجازی‌شده `(virtualized middleware)`: مجموعهٔ مصنوعات زیرساختی‌ای که برای مدیریت و هماهنگ‌کردنِ واحدهای پردازش به کار می‌روند.

شبکهٔ پیام‌رسانی `(messaging grid)`: برای مدیریت درخواست‌های ورودی و حالت نشست `(session state)` به کار می‌رود.

شبکهٔ داده `(data grid)`: هم‌گام‌سازی و تکثیر داده‌ها میان واحدهای پردازش را مدیریت می‌کند.

شبکهٔ پردازش `(processing grid)`: برای مدیریت ارکستراسیونِ درخواست‌ها میان چندین واحد پردازش به کار می‌رود.

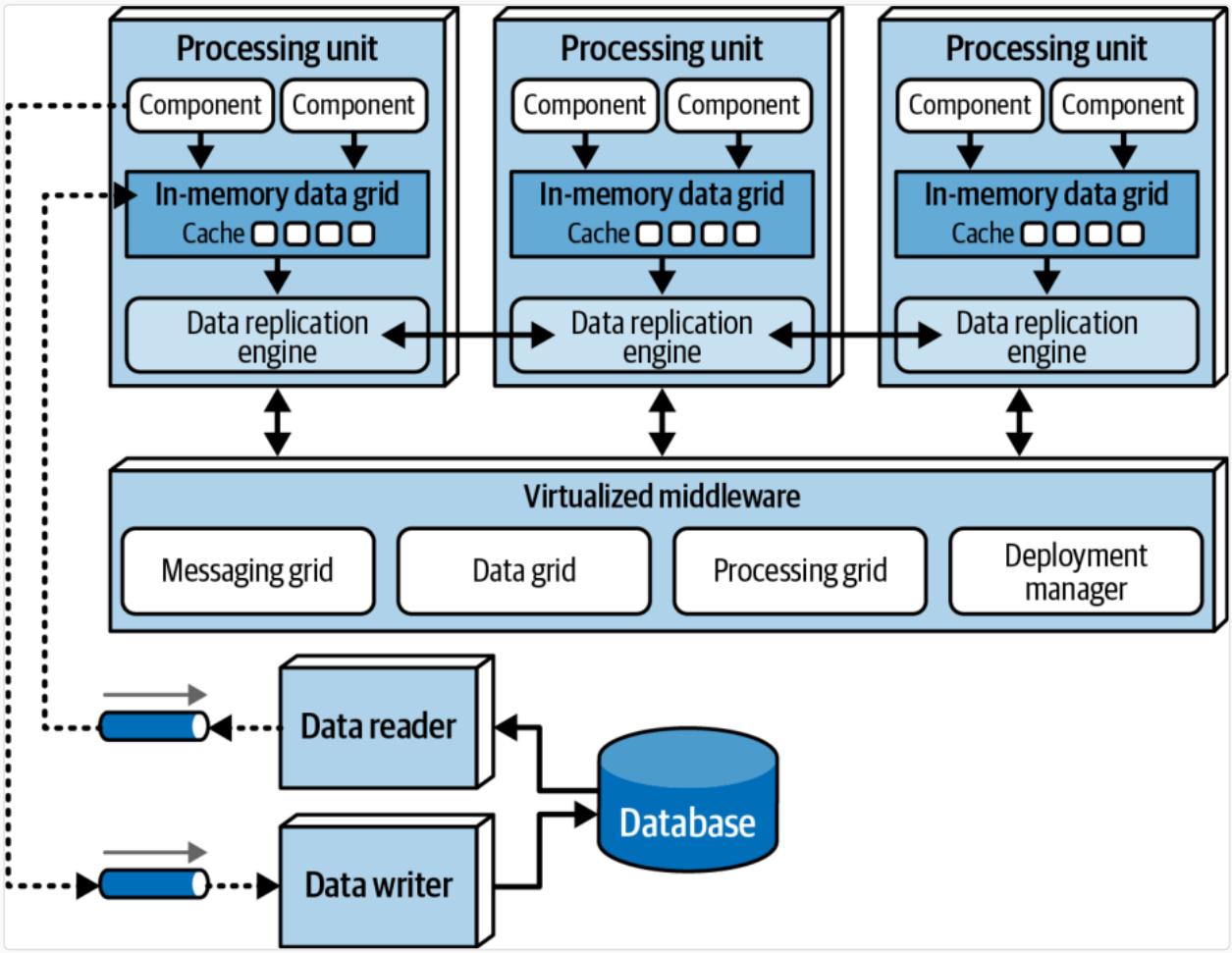
مدیر استقرار `(deployment manager)`: راه‌اندازی و برچیدنِ نمونه‌های واحد پردازش را با افزایش و کاهش بار مدیریت می‌کند.

پمپ‌های داده `(data pumps)`: داده‌های به‌روزشده را به‌صورت ناهم‌گام به پایگاه‌داده می‌فرستند.

نویسنده‌های داده `(data writers)`: به‌روزرسانی‌های دریافتی از پمپ‌های داده را انجام می‌دهند.

خواننده‌های داده `(data readers)`: داده‌های پایگاه‌داده را می‌خوانند و هنگام راه‌اندازیِ واحدهای پردازش آن‌ها را به این واحدها تحویل می‌دهند.

شکل ۱۶-۲ این مصنوعات اصلی را نشان می‌دهد.



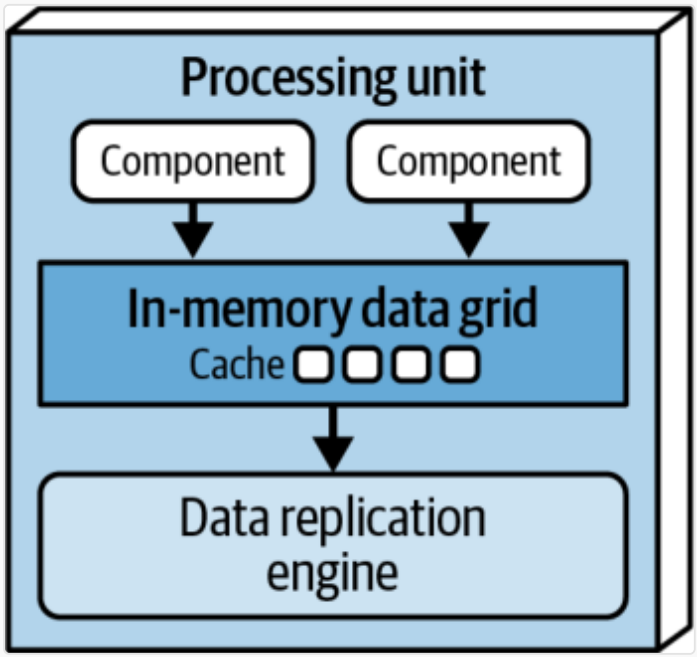
شکل ۱۶-۲. توپولوژی پایه‌ای معماری مبتنی بر فضا

ویژگی‌های خاصِ سبک

بخش‌های پیش رو این مصنوعات اصلی و چگونگیِ کارکردشان را به‌تفصیل شرح می‌دهند.

واحد پردازش

واحد پردازش (که در شکل ۱۶-۳ نشان داده شده) منطق برنامه (یا بخش‌هایی از آن) را در بر دارد که معمولاً هم مؤلفه‌های مبتنی بر وب و هم منطق کسب‌وکارِ پس‌زمینه‌ای را شامل می‌شود. محتوای واحد پردازش بسته به نوع برنامه متفاوت است. برنامه‌های کوچک‌ترِ مبتنی بر وب معمولاً در یک واحد پردازشِ واحد مستقر می‌شوند، در حالی که برنامه‌های بزرگ‌تر اغلب کارکرد خود را بر پایهٔ حوزه‌های کارکردیِ برنامه در چندین واحد پردازش تقسیم می‌کنند. واحد پردازش همچنین می‌تواند سرویس‌های کوچک و تک‌منظوره (کمابیش مانند میکروسرویس‌ها) را در بر داشته باشد. افزون بر این، واحد پردازش یک شبکهٔ دادهٔ درون‌حافظه‌ای و یک موتور تکثیر `(replication engine)` نیز دارد که معمولاً با محصولاتی مانند `Hazelcast`، `Apache Ignite` و `Oracle Coherence` پیاده‌سازی می‌شوند (بخش «شبکهٔ داده» در همین فصل را ببینید).



شکل ۱۶-۳. واحد پردازش کارکرد برنامه را در بر دارد

میان‌افزار مجازی‌شده

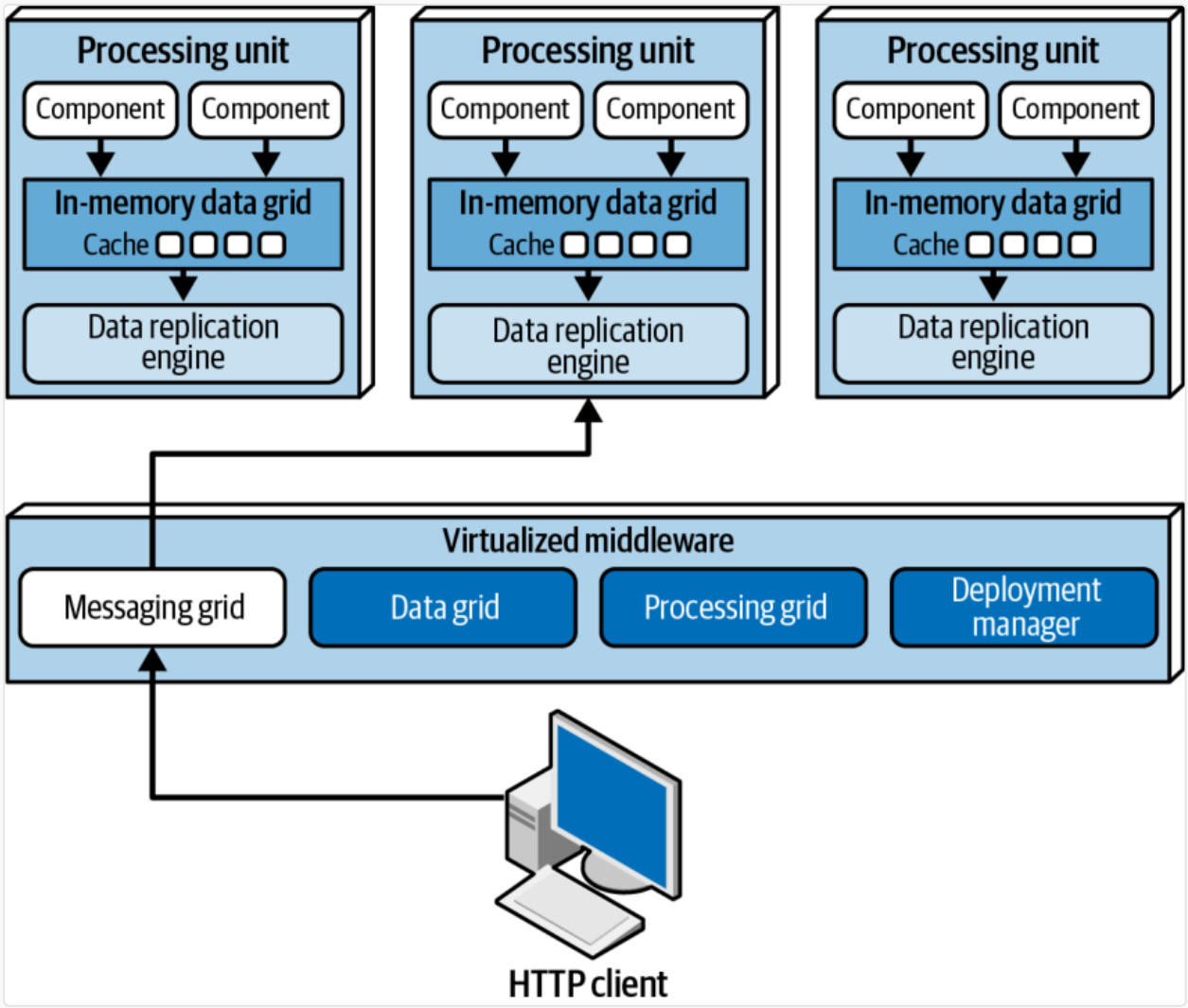
میان‌افزار مجازی‌شده، چنان‌که در شکل ۱۶-۴ نشان داده شده، مصنوعات زیرساختیِ گوناگونی را در بر دارد و برای مدیریت و کنترل واحدهای پردازش به کار می‌رود. این مصنوعِ میان‌افزاری دست‌کم شامل این اجزاست: یک شبکهٔ پیام‌رسانی برای مدیریت درخواست‌های ورودی و حالت نشست کاربر، یک شبکهٔ داده برای مدیریت تکثیر و هم‌گام‌سازیِ داده‌ها، و یک مدیر استقرار برای راه‌اندازی و برچیدنِ نمونه‌های واحد پردازش هر زمان که به آن‌ها نیاز باشد یا نباشد. میان‌افزار مجازی‌شده می‌تواند به‌صورت اختیاری یک شبکهٔ پردازش نیز داشته باشد، برای حالتی که دو یا چند واحد پردازش باید برای یک درخواست کسب‌وکاریِ واحد هماهنگ شوند. معمار می‌تواند هر کارکرد زیرساختیِ دیگری را نیز در صورت نیاز به میان‌افزار مجازی‌شده بیفزاید، مانند کارکردهای امنیتی، گردآوریِ معیارها برای مشاهده‌پذیری و مانند آن.

از آنجا که هیچ محصول واحدی نمی‌تواند همهٔ کارکردهای میان‌افزار مجازی‌شده را انجام دهد، این میان‌افزار معمولاً با محصولات شخص ثالث پیاده‌سازی می‌شود؛ مانند سرورهای وب، ابزارهای کش‌گذاری، توازن‌دهنده‌های بار `(load balancers)`، هماهنگ‌کننده‌های سرویس `(service orchestrators)` و مدیران استقرار، تا پایش، راه‌اندازی و برچیدنِ واحدهای پردازش را مدیریت کنند. هر یک از این مصنوعات میان‌افزاری در بخش‌های پیش رو به‌تفصیل شرح داده می‌شوند.

شبکهٔ پیام‌رسانی

شبکهٔ پیام‌رسانی، که در شکل ۱۶-۴ نشان داده شده، بخشی از میان‌افزار مجازی‌شده است و درخواست‌های ورودی و حالت نشست را مدیریت می‌کند. هنگامی که درخواستی به میان‌افزار مجازی‌شده می‌رسد، مؤلفهٔ شبکهٔ پیام‌رسانی تعیین می‌کند کدام واحدهای پردازشِ فعال برای دریافت آن در دسترس‌اند و سپس درخواست را به یکی از آن واحدهای پردازش هدایت می‌کند.

پیچیدگیِ شبکهٔ پیام‌رسانی می‌تواند متفاوت باشد: از یک الگوریتم نوبت‌گردشیِ `(round-robin)` ساده تا یک الگوریتم پیچیده‌ترِ «نخستین واحد در دسترس» `(next-available)` که ردگیری می‌کند کدام واحد پردازش بیشترین آمادگی را دارد. این مؤلفه معمولاً با یک سرور وبِ متداول که قابلیت‌های توازن‌بار دارد (مانند `HA Proxy` یا `Nginx`) پیاده‌سازی می‌شود.

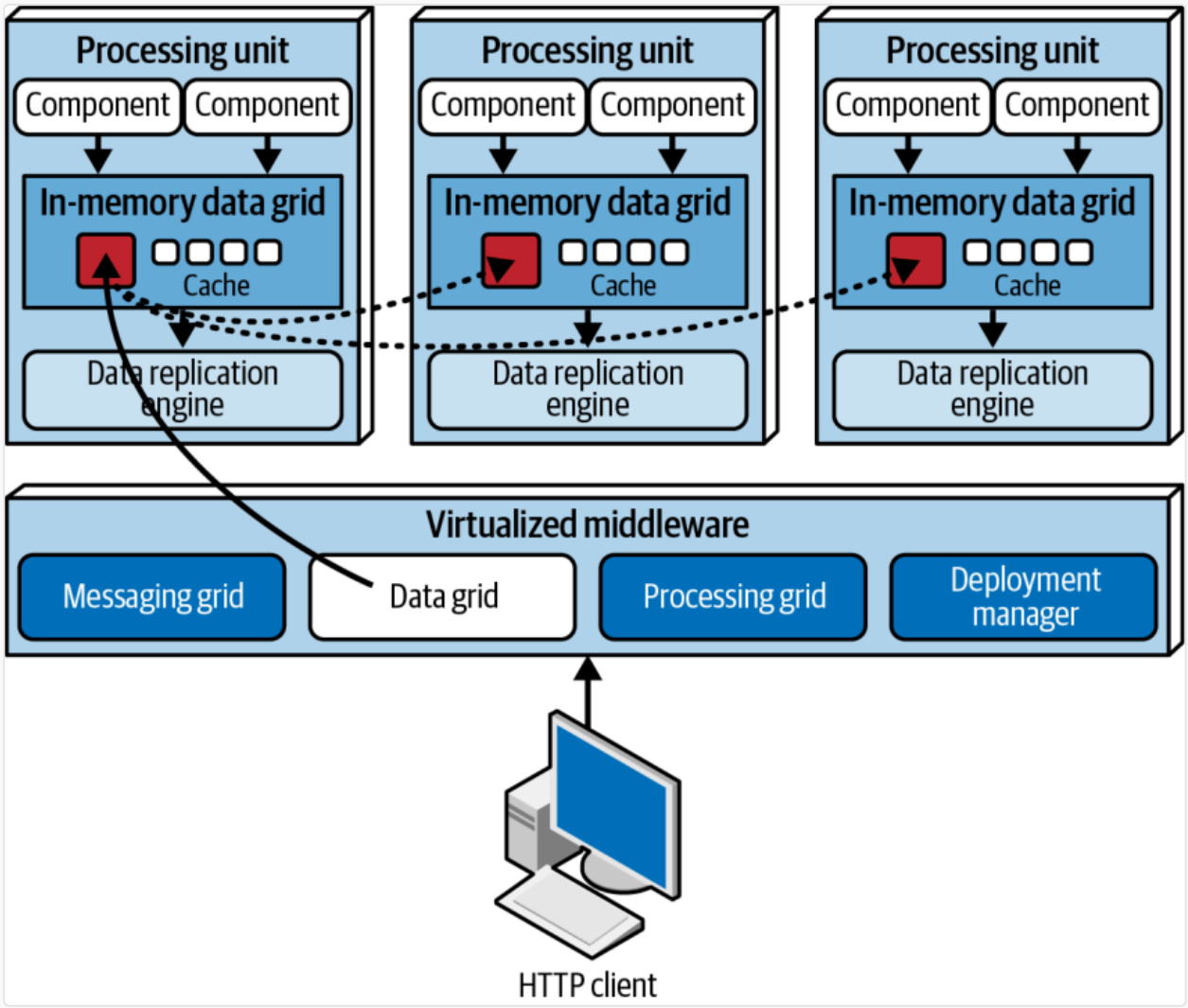


شکل ۱۶-۴. شبکهٔ پیام‌رسانی درخواست‌ها و حالت نشست را رسیدگی می‌کند

شبکهٔ داده

شبکهٔ داده یکی از مؤلفه‌های میان‌افزار مجازی‌شده است، و شاید حیاتی‌ترین مؤلفهٔ آن. در بیشتر پیاده‌سازی‌های مدرن، شبکهٔ داده تنها درون واحدهای پردازش و به‌صورت یک کش تکثیرشدهٔ درون‌حافظه‌ای `(in-memory replicated cache)` پیاده‌سازی می‌شود (بخش «کش‌گذاری تکثیری و توزیع‌شده» در همین فصل را ببینید). اما در پیاده‌سازی‌های کش‌گذاری تکثیری که به یک کنترل‌گر بیرونی `(external controller)` نیاز دارند، یا در پیاده‌سازی‌هایی که از کش توزیع‌شده استفاده می‌کنند، این کارکرد هم در واحدهای پردازش و هم در مؤلفه‌های شبکهٔ دادهٔ میان‌افزار مجازی‌شده جای دارد.

از آنجا که شبکهٔ پیام‌رسانی می‌تواند درخواست را به هر یک از واحدهای پردازشِ در دسترس هدایت کند، ضروری است که شبکهٔ دادهٔ درون‌حافظه‌ایِ هر واحد پردازش دقیقاً همان داده‌ها را در بر داشته باشد. چنان‌که خط‌چین‌ها در شکل ۱۶-۵ نشان می‌دهند، تکثیر داده میان واحدهای پردازش معمولاً به‌صورت ناهم‌گام انجام می‌شود و معمولاً در کمتر از ۱۰۰ میلی‌ثانیه کامل می‌شود.



شکل ۱۶-۵. شبکهٔ داده کش‌های درون‌حافظه‌ای را هم‌گام‌سازی می‌کند

داده‌ها میان واحدهای پردازشی هم‌گام‌سازی می‌شوند که شبکهٔ داده‌ای با نام یکسان دارند. برای نمونه، کد `Java` زیر با استفاده از `Hazelcast` یک شبکهٔ دادهٔ تکثیرشدهٔ داخلی برای واحدهای پردازشی می‌سازد که اطلاعات نمایهٔ مشتری را در بر دارند:

```
HazelcastInstance hz = Hazelcast.newHazelcastInstance();
Map<String, CustomerProfile> profileCache =
        hz.getReplicatedMap("CustomerProfile");
```

همهٔ واحدهای پردازشی که به اطلاعات نمایهٔ مشتری دسترسی لازم دارند باید این کد را داشته باشند. هنگامی که هر واحد پردازشی داده‌های کش `CustomerProfile` را به‌روز می‌کند، شبکهٔ داده آن به‌روزرسانی را در همهٔ واحدهای پردازشِ دیگری که کشی با همان نام `CustomerProfile` دارند تکثیر می‌کند. یک واحد پردازش می‌تواند به هر تعداد کش تکثیرشدهٔ درون‌حافظه‌ای که برای انجام کارش لازم دارد مجهز باشد. راه دیگر این است که یک واحد پردازش برای درخواست داده، فراخوانی دوری `(remote call)` به واحد پردازشِ دیگری انجام دهد (کرئوگرافی) یا از شبکهٔ پردازش (که در بخش بعد شرح داده می‌شود) برای ارکستراسیونِ درخواست داده بهره بگیرد (برای اطلاعات بیشتر دربارهٔ ارکستراسیون و کرئوگرافی، بخش «ارکستراسیون در برابر کرئوگرافی» در فصل ۲۰ را ببینید).

استفاده از تکثیر داده درون واحدهای پردازش، به شرط آنکه دست‌کم یک نمونه کشِ تکثیرشدهٔ نام‌دار را در بر داشته باشد، امکان می‌دهد نمونه‌های بیشتری بدون نیاز به خواندن داده از پایگاه‌داده راه‌اندازی شوند. هنگامی که نمونهٔ تازه‌ای از یک واحد پردازش راه‌اندازی می‌شود، از راه ارائه‌دهندهٔ کش `(caching provider)` (مانند `Hazelcast`) درخواستی را پخش `(broadcast)` می‌کند تا به دیگر واحدهای پردازشی که کشی با همان نام دارند بپیوندد. همین که دیگر واحدهای پردازش درخواست پخش‌شده را تأیید کنند و به واحد پردازش تازه متصل شوند، یکی از آن‌ها (معمولاً نخستین واحدی که به واحد پردازش تازه متصل می‌شود) داده‌های کش را برای نمونهٔ تازه می‌فرستد تا با همهٔ نمونه‌های دیگری که کشی با همان نام دارند هم‌گام باشد.

هر نمونه از یک واحد پردازش یک فهرست اعضا `(member list)` دارد که نشانی‌های `IP` و درگاه‌های `(ports)` همهٔ نمونه‌های دیگرِ واحد پردازش را که کشی با همان نام دارند در بر دارد. برای نمونه، فرض کنید تنها یک نمونهٔ واحد پردازش وجود دارد که کارکرد نمایهٔ مشتری و داده‌های کش‌شدهٔ تکثیریِ درون‌حافظه‌ای‌اش را در بر دارد. در این حالت تنها یک نمونه وجود دارد، پس فهرست اعضایش تنها خودش را در بر دارد. این وضعیت در عبارت‌های ثبت وقایعِ زیر که با `Hazelcast` تولید شده‌اند نشان داده شده است:

```
Instance 1:
Members {size:1, ver:1} [
        Member [172.19.248.89]:5701 - 04a6f863-dfce-41e5-9d51-9f4e356ef268 this
]
```

هنگامی که واحد پردازشِ دیگری با کشی به همان نام راه‌اندازی می‌شود، فهرست اعضای هر دو سرویس به‌روز می‌شود تا نشانی‌های `IP` و درگاه‌های هر دو واحد پردازش را بازتاب دهد:

```
Instance 1:
Members {size:2, ver:2} [
        Member [172.19.248.89]:5701 - 04a6f863-dfce-41e5-9d51-9f4e356ef268 this
        Member [172.19.248.90]:5702 - ea9e4dd5-5cb3-4b27-8fe8-db5cc62c7316
]

Instance 2:
Members {size:2, ver:2} [
        Member [172.19.248.89]:5701 - 04a6f863-dfce-41e5-9d51-9f4e356ef268
        Member [172.19.248.90]:5702 - ea9e4dd5-5cb3-4b27-8fe8-db5cc62c7316 this
]
```

هنگامی که واحد پردازش سوم راه‌اندازی می‌شود، فهرست اعضای نمونهٔ ۱ و نمونهٔ ۲ هر دو به‌روز می‌شوند تا نمونهٔ سومِ تازه را بازتاب دهند:

```
Instance 1:
Members {size:3, ver:3} [
        Member [172.19.248.89]:5701 - 04a6f863-dfce-41e5-9d51-9f4e356ef268 this
        Member [172.19.248.90]:5702 - ea9e4dd5-5cb3-4b27-8fe8-db5cc62c7316
        Member [172.19.248.91]:5703 - 1623eadf-9cfb-4b83-9983-d80520cef753
]

Instance 2:
Members {size:3, ver:3} [
        Member [172.19.248.89]:5701 - 04a6f863-dfce-41e5-9d51-9f4e356ef268
        Member [172.19.248.90]:5702 - ea9e4dd5-5cb3-4b27-8fe8-db5cc62c7316 this
        Member [172.19.248.91]:5703 - 1623eadf-9cfb-4b83-9983-d80520cef753
]

Instance 3:
Members {size:3, ver:3} [
        Member [172.19.248.89]:5701 - 04a6f863-dfce-41e5-9d51-9f4e356ef268
        Member [172.19.248.90]:5702 - ea9e4dd5-5cb3-4b27-8fe8-db5cc62c7316
        Member [172.19.248.91]:5703 - 1623eadf-9cfb-4b83-9983-d80520cef753 this
]
```

اکنون هر سه نمونه از یکدیگر آگاه‌اند (از جمله از خودشان، که با واژهٔ `this` در انتهای سطر عضو مشخص شده است). فرض کنید نمونهٔ ۱ درخواستی از یک مشتری برای به‌روزرسانیِ نشانیِ صورت‌حسابش دریافت می‌کند. هنگامی که نمونهٔ ۱ کش را با `cache.put()` یا روش مشابهی برای به‌روزرسانی کش به‌روز می‌کند، شبکهٔ داده (مانند `Hazelcast`) کش‌های تکثیرشدهٔ دیگر را به‌صورت ناهم‌گام با همان به‌روزرسانی به‌روز می‌کند و اطمینان می‌دهد که هر سه کشِ نمایهٔ مشتری نشانیِ صورت‌حسابِ تازه را در بر دارند و بدین ترتیب همواره با داده‌های یکسان با هم هم‌گام می‌مانند.

هنگامی که نمونه‌های واحد پردازش از کار می‌افتند، فهرست اعضای همهٔ واحدهای پردازشِ دیگر به‌طور خودکار به‌روز می‌شود تا این تغییر را بازتاب دهد. برای نمونه، اگر نمونهٔ ۲ از کار بیفتد، محصول کش‌گذاری بی‌درنگ فهرست اعضای نمونه‌های ۱ و ۳ را به‌روز می‌کند تا نمونهٔ ازدست‌رفته را حذف کند:

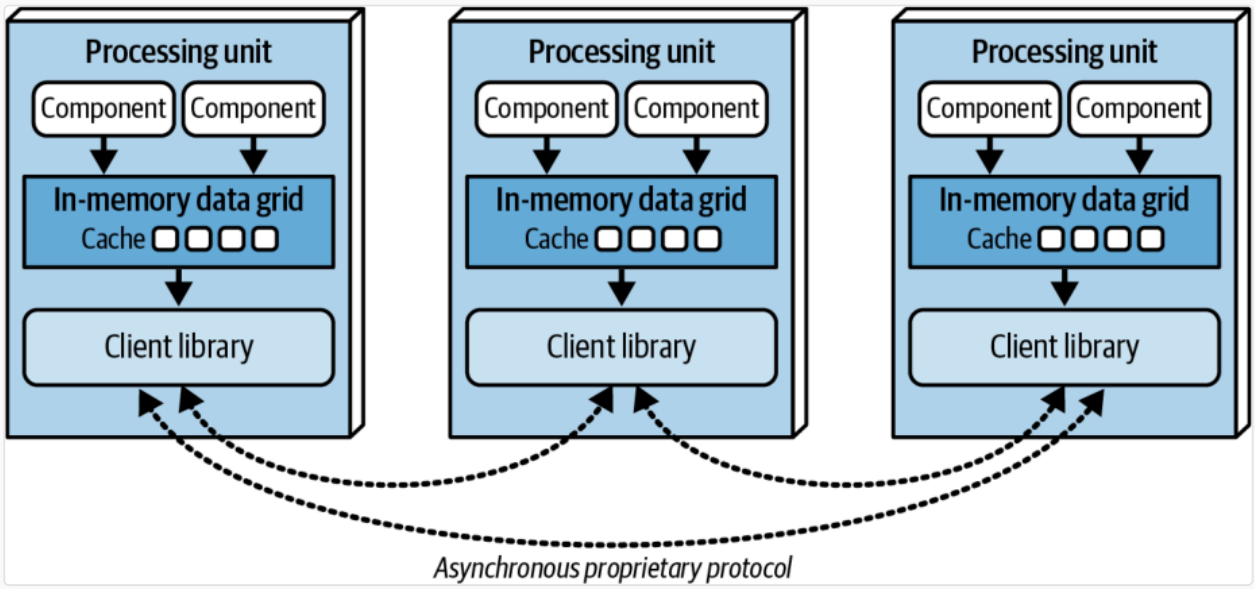
```
Instance 1:
Members {size:2, ver:4} [
        Member [172.19.248.89]:5701 - 04a6f863-dfce-41e5-9d51-9f4e356ef268 this
        Member [172.19.248.91]:5703 - 1623eadf-9cfb-4b83-9983-d80520cef753
]

Instance 3:
Members {size:2, ver:4} [
        Member [172.19.248.89]:5701 - 04a6f863-dfce-41e5-9d51-9f4e356ef268
        Member [172.19.248.91]:5703 - 1623eadf-9cfb-4b83-9983-d80520cef753 this
]
```

کش‌گذاری تکثیری و توزیع‌شده

معماری مبتنی بر فضا برای پردازش تراکنش‌ها در برنامه‌ها به کش‌گذاری متکی است. این معماری نیاز به خواندن و نوشتنِ مستقیم در پایگاه‌داده را از میان برمی‌دارد و همین است که به آن امکان می‌دهد از مقیاس‌پذیری، کشسانی و کارایی بالا پشتیبانی کند. این سبک معماری بیشتر به کش‌گذاری تکثیریِ درون‌حافظه‌ای `(in-memory replicated caching)` متکی است، هرچند می‌تواند از کش‌گذاری توزیع‌شده `(distributed caching)` نیز استفاده کند.

در کش‌گذاری تکثیری، چنان‌که در شکل ۱۶-۶ نشان داده شده، هر واحد پردازش شبکهٔ دادهٔ درون‌حافظه‌ایِ خود را دارد که میان همهٔ واحدهای پردازشی که از کشی با همان نام استفاده می‌کنند هم‌گام‌سازی می‌شود. هنگامی که کشی درون هر یک از واحدهای پردازش به‌روز می‌شود، واحدهای پردازشِ دیگر به‌طور خودکار با اطلاعات تازه به‌روز می‌شوند.



شکل ۱۶-۶. کش‌گذاری تکثیری کش‌های درون‌حافظه‌ای را میان واحدهای پردازش هم‌گام‌سازی می‌کند

کش‌گذاری تکثیری نه‌تنها بسیار سریع است، بلکه از سطح بالایی از تحمل خطا نیز پشتیبانی می‌کند. از آنجا که هیچ سرور مرکزی‌ای کش را نگه نمی‌دارد، کش‌گذاری تکثیری نقطهٔ شکست واحد ندارد [پانویس ۱].

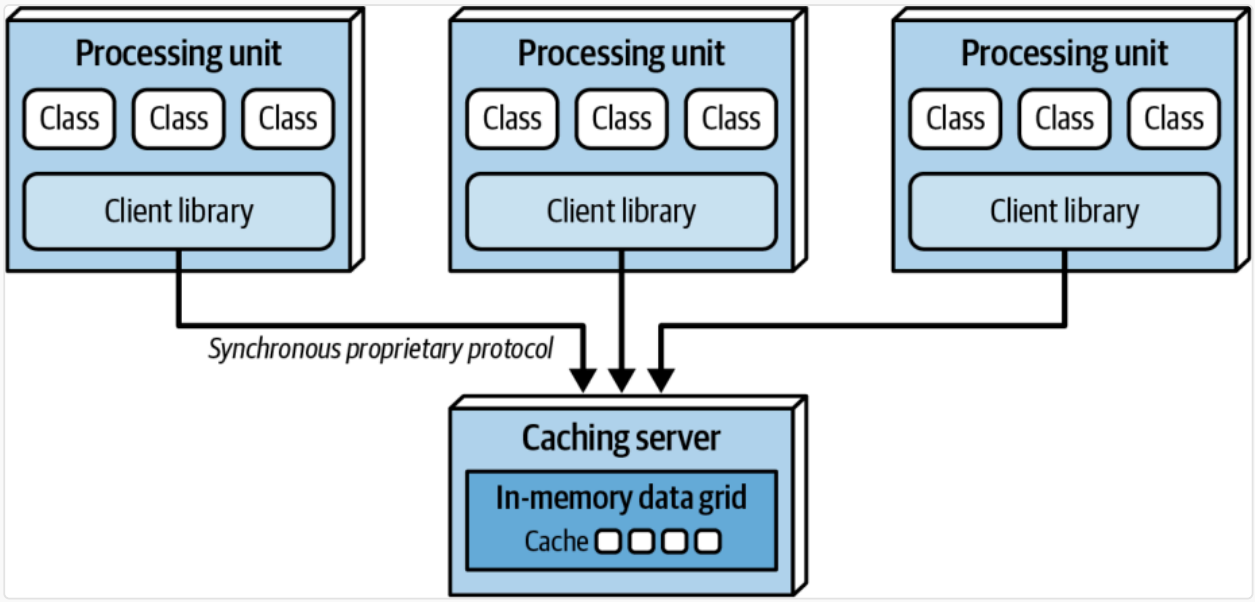
> پانویس ۱ – بسته به پیاده‌سازیِ محصول کش‌گذاریِ مورد استفاده، ممکن است استثناهایی بر این قاعده وجود داشته باشد. برخی محصولات کش‌گذاری برای پایش و کنترل تکثیر داده میان واحدهای پردازش به یک کنترل‌گر بیرونی نیاز دارند، اما بیشترشان در حال کنارگذاشتنِ این مدل‌اند.

هرچند کش‌گذاری تکثیری مدل کش‌گذاریِ استانداردِ معماری مبتنی بر فضاست، مواردی وجود دارد که نمی‌توان از آن استفاده کرد. یکی از این موقعیت‌ها زمانی است که سیستم باید حجم بالایی از داده `(high data volumes)` را رسیدگی کند. هنگامی که کش‌های حافظهٔ داخلی از ۱۰۰ مگابایت بزرگ‌تر می‌شوند، هر واحد پردازش ناچار است چنان حافظهٔ زیادی مصرف کند که ممکن است در کشسانی و مقیاس‌پذیری مشکل پدید آید. واحدهای پردازش معمولاً درون

یک ماشین مجازی (virtual machine) یا یک کانتینر (مانند Docker) مستقر می‌شوند که هر یک تنها مقدار مشخصی حافظه برای استفادهٔ کش داخلی در اختیار دارد. این امر تعداد نمونه‌های واحد پردازش را که می‌توان برای پردازش موقعیت‌های پرتوان (high-throughput) راه‌اندازی کرد محدود می‌کند.

موقعیت دیگری که کش‌های دادهٔ تکثیری در آن خوب کار نمی‌کنند، زمانی است که داده‌های کش بسیار پرتکرار به‌روز می‌شوند. چنان‌که در بخش «برخوردهای داده» در همین فصل نشان داده شده است، اگر نرخ به‌روزرسانیِ داده‌های کش بیش از اندازه بالا باشد، شبکهٔ داده ممکن است نتواند پابه‌پای آن پیش برود و این امر می‌تواند به سازگاری داده در همهٔ نمونه‌های واحد پردازش آسیب بزند. هنگامی که چنین موقعیت‌هایی پیش می‌آید، بیشتر معماران استفاده از کش توزیع‌شده را برمی‌گزینند.

کش‌گذاری توزیع‌شده، چنان‌که در شکل ۱۶-۷ نشان داده شده، به یک سرور یا سرویس بیرونیِ اختصاصی برای نگهداریِ یک کش متمرکز نیاز دارد. در این مدل، واحدهای پردازش داده‌ها را در حافظهٔ داخلی ذخیره نمی‌کنند. در عوض، برای دسترسی به داده‌ها در سرور کشِ مرکزی از یک پروتکل اختصاصیِ فروشنده (proprietary protocol) استفاده می‌کنند. کش‌گذاری توزیع‌شده از سطح بالایی از سازگاری داده پشتیبانی می‌کند، زیرا همهٔ داده‌ها در یک جا نگه داشته می‌شوند و نیازی به تکثیر ندارند. اما کاراییِ این مدل به خوبیِ کش‌گذاری تکثیری نیست، زیرا داده‌های کش باید از راه دور خوانده شوند و همین امر بر تأخیر کلیِ سیستم می‌افزاید.



شکل ۱۶-۷. کش‌گذاری توزیع‌شده سازگاری دادهٔ خوبی میان واحدهای پردازش پدید می‌آورد

تحمل خطا نیز در کش‌گذاری توزیع‌شده یک مسئله است. اگر سرور کشی که داده‌ها را در بر دارد از کار بیفتد، هیچ‌یک از واحدهای پردازش نمی‌توانند به داده‌ها دسترسی داشته باشند یا آن‌ها را به‌روز کنند. می‌توان این نقطهٔ شکست واحد را با آینه‌سازی (mirroring) کش توزیع‌شده کاهش داد، اما اگر سرور کش اصلی به‌طور غیرمنتظره از کار بیفتد و داده‌ها به سرور کشِ آینه نرسند، این کار ممکن است مشکلات سازگاری پدید آورد.

هنگامی که اندازهٔ کش نسبتاً کوچک است (زیر ۱۰۰ مگابایت) و نرخ به‌روزرسانیِ کش به اندازه‌ای پایین است که موتور تکثیرِ محصول کش‌گذاری بتواند پابه‌پای آن پیش برود، تصمیم‌گیری میان استفاده از کش تکثیری و کش توزیع‌شده به پرسشی دربارهٔ اولویت‌بندیِ سازگاری داده در برابر کارایی و تحمل خطا تبدیل می‌شود. کش توزیع‌شده همواره سازگاری دادهٔ بهتری نسبت به کش تکثیری فراهم می‌کند، زیرا داده‌ها در یک جا قرار دارند و میان چندین واحد پردازش پخش نشده‌اند. اما کارایی و تحمل خطا همواره با کش تکثیری بهتر خواهد بود. بسیاری اوقات، عامل تعیین‌کننده نوع داده‌ای است که در واحدهای پردازش کش می‌شود. اگر نیاز اصلیِ سیستم به داده‌هایی با سازگاری بالا باشد (مانند شمار موجودیِ محصولات در دسترس)، این امر معمولاً استفاده از کش توزیع‌شده را توجیه می‌کند. اگر داده‌ها اغلب تغییر نکنند (مانند دادهٔ مرجع `(reference data)` از قبیل جفت‌های نام/مقدار، کدهای محصول و توضیحات محصول)، استفاده از کش تکثیری را برای جست‌وجوی سریع در نظر بگیرید. جدول ۱۶-۱ برخی معیارهای انتخاب میان کش توزیع‌شده و کش تکثیری را خلاصه می‌کند.

جدول ۱۶-۱. کش‌گذاری توزیع‌شده در برابر کش‌گذاری تکثیری

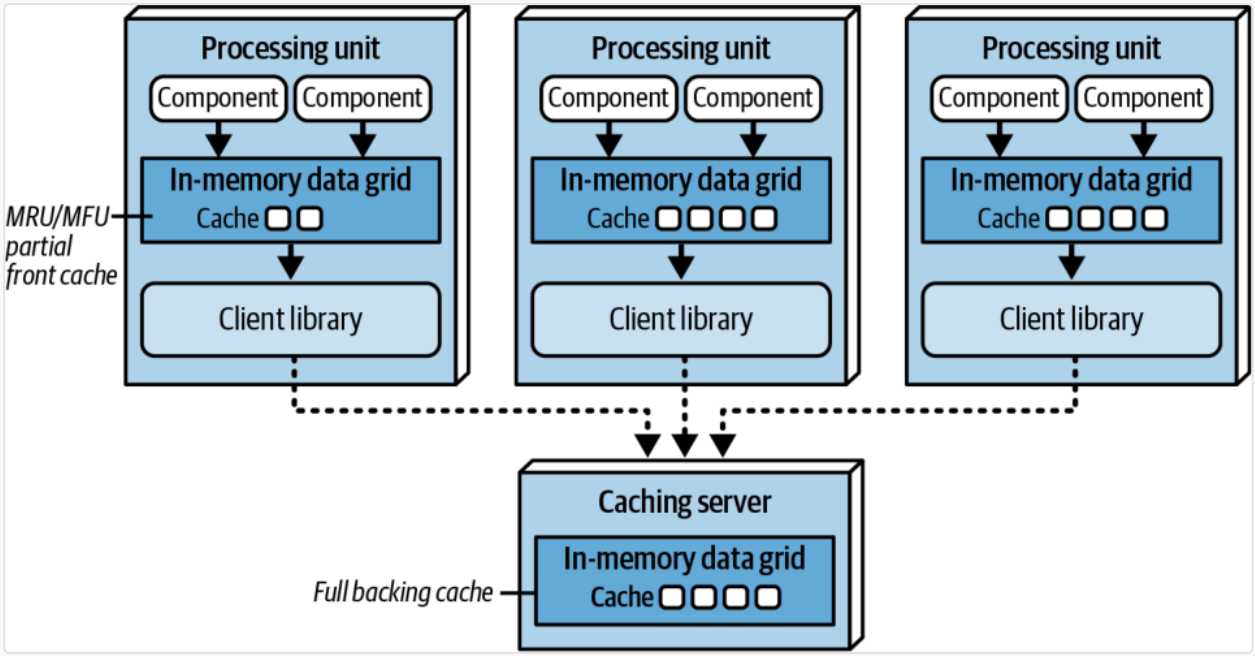
معیار تصمیم‌گیری	کش تکثیری	کش توزیع‌شده
بهینه‌سازی	کارایی	سازگاری
اندازهٔ کش	کوچک (کمتر از ۱۰۰ مگابایت)	بزرگ (بیش از ۵۰۰ مگابایت)
نوع داده	نسبتاً ایستا	به‌شدت پویا
بسامد به‌روزرسانی	نسبتاً پایین	نرخ به‌روزرسانیِ بالا
تحمل خطا	بالا	پایین

هنگام انتخاب مدل کش‌گذاری در معماری مبتنی بر فضا، به یاد داشته باشید که در بیشتر موارد هر دو مدل کاربردپذیرند. به بیان دیگر، نه کش‌گذاری تکثیری و نه کش‌گذاری توزیع‌شده هیچ‌کدام همهٔ مسائل را حل نمی‌کنند. واحدهای پردازشِ گوناگون نیز می‌توانند از مدل‌های گوناگون استفاده کنند. به‌جای آنکه با برگزیدنِ یک مدل کش‌گذاریِ یکسان در سراسر برنامه کوتاه بیایید، از هر مدل برای نقاط قوتش بهره بگیرید. برای نمونه، برای واحد پردازشی که موجودیِ فعلی را نگه می‌دارد، مدل کش‌گذاری توزیع‌شده را به خاطر سازگاری داده برگزینید؛ و برای واحد پردازشی که نمایهٔ مشتری را نگه می‌دارد، کش تکثیری را به خاطر کارایی و تحمل خطا انتخاب کنید.

ملاحظات کش نزدیک

کش نزدیک `(near-cache)` یک مدل کش‌گذاریِ ترکیبی است که شبکه‌های دادهٔ درون‌حافظه‌ای را با یک کش توزیع‌شده پیوند می‌دهد. در این مدل (که در شکل ۱۶-۸ نشان داده شده)، کش توزیع‌شده کش پشتیبانِ کامل `(full backing cache)` نامیده می‌شود و هر شبکهٔ دادهٔ درون‌حافظه‌ای که درون یک واحد پردازش جای دارد، کش جلویی `(front cache)` نام دارد. کش جلویی همواره زیرمجموعهٔ کوچک‌تری از کش پشتیبانِ کامل را در بر دارد و از یک سیاست بیرون‌رانی `(eviction policy)` استفاده می‌کند تا اقلام قدیمی‌تر را حذف کند و جا برای افزودنِ اقلام تازه‌تر

باز شود. کش جلویی سه گزینهٔ سیاست بیرون‌رانی دارد: کش «تازه‌ترین استفاده» (most recently used) که اقلامی را در بر دارد که از همه تازه‌تر به کار رفته‌اند؛ کش «پرتکرارترین استفاده» (most frequently used) که اقلامی را در بر دارد که بیش از همه به کار رفته‌اند؛ یا سیاست بیرون‌رانیِ جایگزینیِ تصادفی (random replacement) که هر گاه به فضا نیاز باشد، اقلام را به‌طور تصادفی حذف می‌کند. جایگزینیِ تصادفی سیاست بیرون‌رانیِ خوبی است، هنگامی که هیچ تحلیل روشنی از داده‌ها دلیلی برای نگه‌داشتنِ اقلامِ اخیراً به‌کاررفته یا پرتکرارترین اقلام به دست نمی‌دهد.

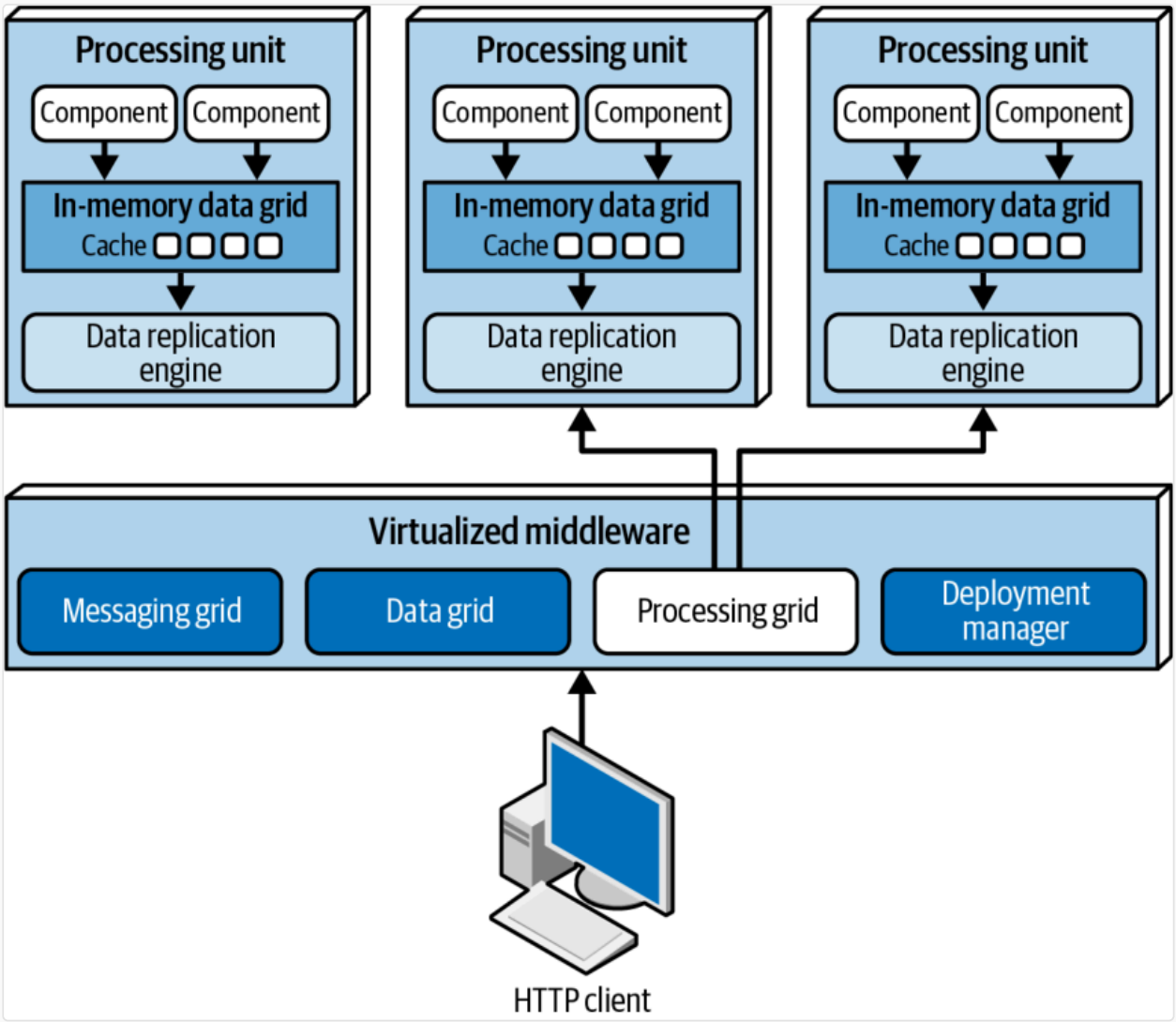


شکل ۱۶-۸. مدل کش نزدیک هم از کش جلویی و هم از کش پشتیبان استفاده می‌کند

هرچند کش‌های جلویی همواره با کش پشتیبانِ کامل هم‌گام نگه داشته می‌شوند، کش‌های جلوییِ درون هر واحد پردازش با کش‌های واحدهای پردازشِ دیگری که همان داده‌ها را به اشتراک می‌گذارند هم‌گام‌سازی نمی‌شوند. این یعنی چندین واحد پردازش که بافتار دادهٔ یکسانی (مانند نمایهٔ مشتری) را به اشتراک می‌گذارند، به احتمال زیاد هر کدام داده‌های متفاوتی در کش جلوییِ خود خواهند داشت. این امر میان واحدهای پردازش ناهمسانی‌هایی در کارایی و پاسخ‌گویی پدید می‌آورد، از این رو استفاده از مدل کش نزدیک را در معماری مبتنی بر فضا توصیه نمی‌کنیم.

شبکهٔ پردازش

شبکهٔ پردازش، که در شکل ۱۶-۹ نشان داده شده، مؤلفه‌ای اختیاری درون میان‌افزار مجازی‌شده است که هر گاه چندین واحد پردازش در یک درخواست کسب‌وکاریِ واحد دخالت داشته باشند، پردازش درخواستِ ارکسترەشده را مدیریت می‌کند. اگر درخواستی به هماهنگی میان بیش از یک نوع واحد پردازش نیاز داشته باشد (مانند یک واحد پردازش سفارش و یک واحد پردازش پرداخت)، شبکهٔ پردازش میان آن دو واحد پردازش میانجی‌گری می‌کند و درخواست را ارکستره می‌کند.



شکل ۱۶-۹. شبکهٔ پردازش ارکستراسیون میان واحدهای پردازش را مدیریت می‌کند

بیشتر پیاده‌سازی‌های مدرنِ مبتنی بر فضا (به‌ویژه آن‌هایی که سرویس‌های ریزدانه دارند) کارکرد شبکهٔ پردازش را به‌جای یک موتور ارکستراسیونِ `(orchestration engine)` درشت‌دانهٔ واحد، از راه واحدهای پردازشِ ارکستراسیونِ جداگانه و ریزدانه پیاده‌سازی می‌کنند، به گونه‌ای که هر واحد پردازشِ ارکستراسیون یک گردش‌کارِ اصلیِ واحد را رسیدگی کند. برای یک نمونهٔ تجارت الکترونیک، فرض کنید وقتی مشتری سفارشی ثبت می‌کند، سه واحد پردازش باید با هم هماهنگ شوند: ثبت سفارش، پرداخت و تعدیل موجودی. معمار می‌تواند یک واحد پردازشِ «هماهنگ‌کنندهٔ ثبت سفارش» بسازد تا این سه واحد پردازش را ارکستره کند. او همچنین ممکن است واحدهای پردازشِ ارکستراسیونِ جداگانه‌ای برای دیگر گردش‌کارهای اصلی بسازد، مانند رسیدگی به سفارش‌های مرجوعی و تأمین دوبارهٔ موجودیِ انبار.

مدیر استقرار

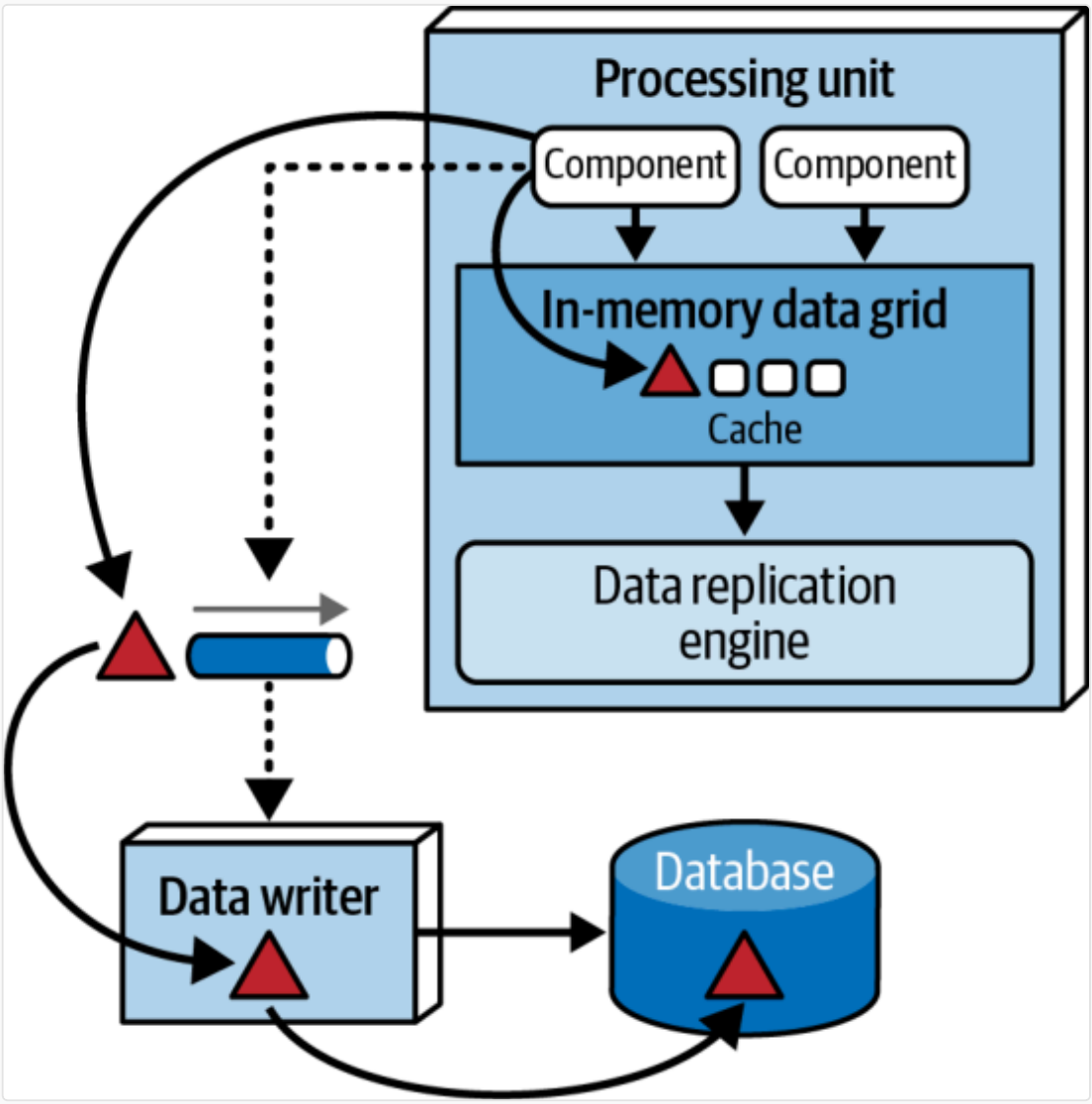
مؤلفهٔ `Deployment Manager` راه‌اندازی و خاموش‌کردنِ پویای نمونه‌های واحد پردازش را بر پایهٔ شرایط بار مدیریت می‌کند. این مؤلفه پیوسته زمان‌های پاسخ `(response times)` و بار کاربران را پایش می‌کند، هنگام افزایش بار واحدهای پردازشِ تازه‌ای راه‌اندازی می‌کند و هنگام کاهش بار واحدهای پردازش را خاموش می‌کند. این مؤلفه برای

دستیابی به مقیاس‌پذیریِ متغیر (کشسانی) در یک برنامه حیاتی است. بیشتر زیرساخت‌های مبتنی بر ابر این مسئولیت را بر عهده می‌گیرند، همان‌گونه که محصولات ارکستراسیون سرویس مانند `Kubernetes` نیز این کار را انجام می‌دهند.

پمپ‌های داده

پمپ داده راهی برای فرستادنِ داده به پردازندهٔ دیگری است که آن پردازنده سپس پایگاه‌داده را به‌روز می‌کند. معماری‌های مبتنی بر فضا به پمپ داده نیاز دارند، زیرا واحدهای پردازش مستقیماً از پایگاه‌داده نمی‌خوانند و در آن نمی‌نویسند. پمپ‌های داده در معماری مبتنی بر فضا همواره ناهم‌گام‌اند و سازگاری نهایی میان کش درون‌حافظه‌ای و پایگاه‌داده را فراهم می‌کنند. هنگامی که یک نمونهٔ واحد پردازش درخواستی دریافت می‌کند و کش خود را به‌روز می‌کند، آن واحد پردازش مالکِ آن به‌روزرسانی می‌شود و از این رو مسئول است آن را از راه پمپ داده بفرستد تا پایگاه‌داده سرانجام به‌روز شود.

پمپ‌های داده در معماری مبتنی بر فضا معمولاً با پیام‌رسانی پیاده‌سازی می‌شوند؛ چنان‌که در شکل ۱۶-۱۰ نشان داده شده است.



شکل ۱۶-۱۰. پمپ‌های داده برای فرستادنِ داده به پایگاه‌داده به کار می‌روند

پیام‌رسانی نه‌تنها از ارتباط ناهم‌گام، بلکه از تحویل تضمین‌شده، ماندگاری پیام (message persistence) و ترتیب پیام (message order) نیز، از راه صف‌بندیِ «نخست وارد، نخست خارج» (FIFO)، پشتیبانی می‌کند. افزون بر این، پیام‌رسانی واحد پردازش و نویسندهٔ داده را از هم جدا می‌کند، به گونه‌ای که اگر نویسندهٔ داده در دسترس نباشد، پردازش درون واحدهای پردازش قطع نمی‌شود.

بیشتر معماری‌های مبتنی بر فضا چندین پمپ داده دارند. معمولاً هر پمپ به یک حوزه یا زیرحوزهٔ مشخص (مانند مشتری یا موجودی) اختصاص دارد، اما می‌توان آن‌ها را به هر نوع کش (مانند CustomerProfile، CustomerWishlist و مانند آن) یا به یک حوزهٔ واحد پردازش (مانند Customer) که کش عمومیِ بسیار بزرگ‌تری نیز دارد اختصاص داد.

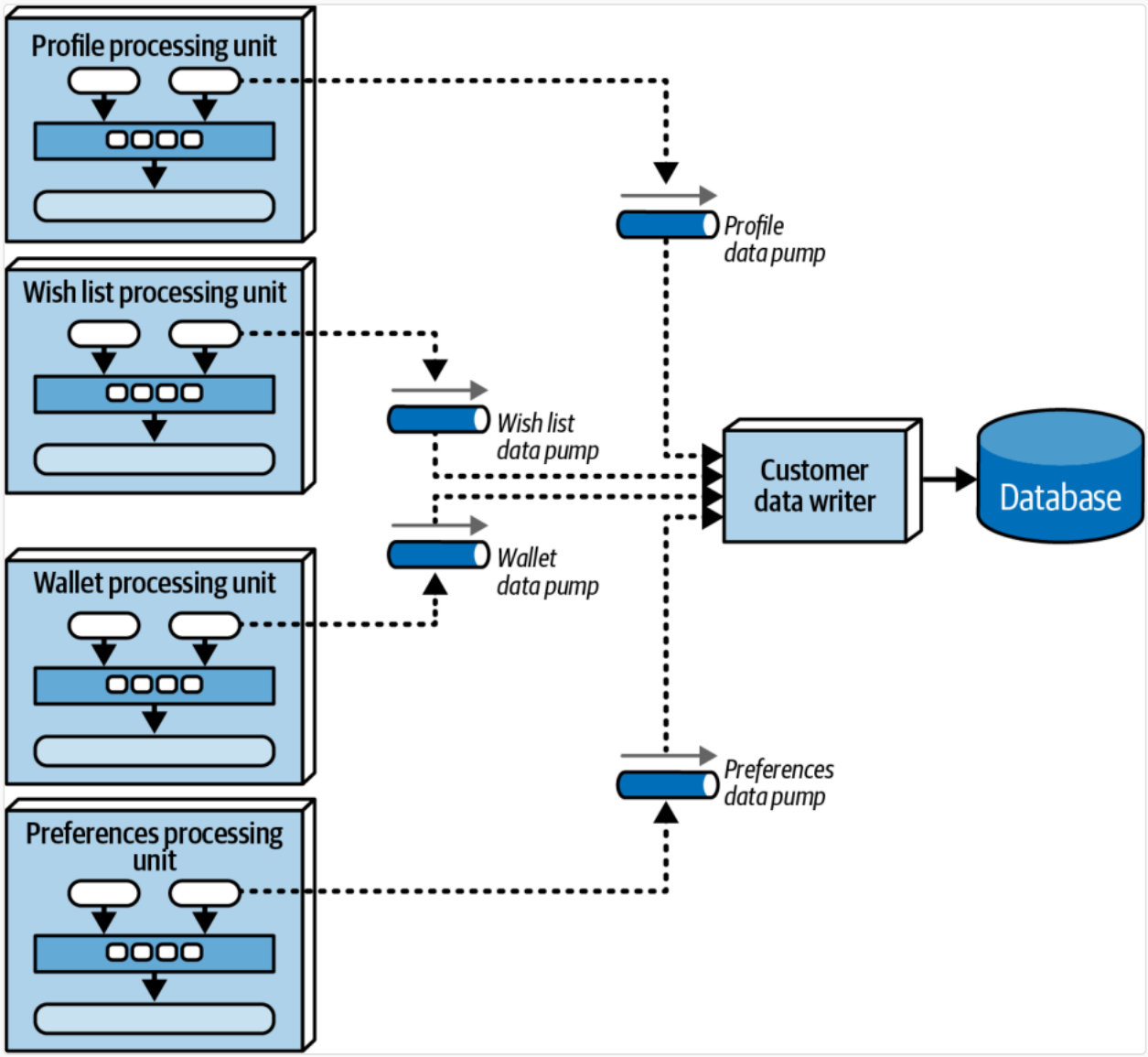
پمپ‌های داده معمولاً قرارداد دارند که شامل یک کنش مرتبط با داده‌های قرارداد (افزودن، حذف یا به‌روزرسانی) است. قرارداد می‌تواند یک طرح‌وارهٔ JSON، یک طرح‌وارهٔ XML، یک شیء یا حتی یک پیام ارزش‌محور (value-driven message) باشد (یعنی یک پیام نگاشتی (map message) که جفت‌های نام-مقدار را در بر دارد). در به‌روزرسانی‌ها،

محمولهٔ پیامِ پمپ داده معمولاً تنها مقادیر تازهٔ داده را در بر دارد. برای نمونه، اگر مشتری شمارهٔ تلفن نمایهٔ خود را تغییر دهد، تنها شمارهٔ تلفن تازه، همراه با شناسهٔ مشتری و کنشی برای به‌روزرسانیِ داده، فرستاده می‌شود.

نویسنده‌های داده

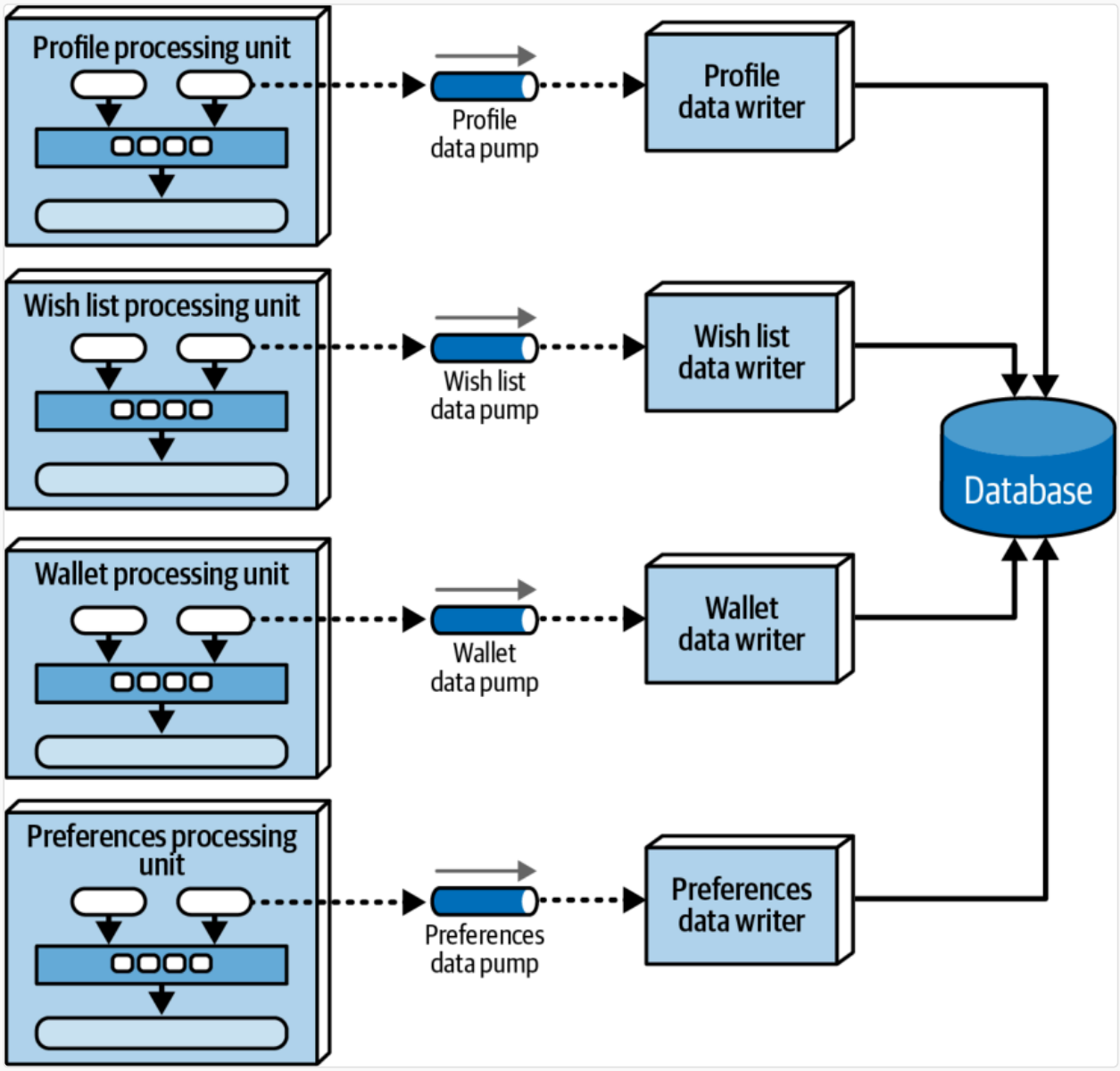
مؤلفهٔ `Data Writer` پیام‌ها را از یک پمپ داده می‌پذیرد و پایگاه‌داده را با اطلاعاتِ محمولهٔ آن‌ها به‌روز می‌کند (شکل ۱۶-۱۰ را ببینید). نویسنده‌های داده را می‌توان به‌صورت سرویس، برنامهٔ کاربردی یا هاب داده `(data hub)` (مانند `Ab Initio`) پیاده‌سازی کرد. دانه‌بندیِ آن‌ها بسته به دامنهٔ پمپ‌های داده و واحدهای پردازش می‌تواند متفاوت باشد.

نویسندهٔ دادهٔ حوزه‌محور `(domain-based data writer)` منطق پایگاه‌داده‌ایِ لازم برای رسیدگی به همهٔ به‌روزرسانی‌های درون یک حوزهٔ مشخص (مانند پردازش سفارش) را در بر دارد، صرف‌نظر از اینکه از چند پمپ داده می‌خواند. سیستم شکل ۱۶-۱۱ چهار واحد پردازشِ متفاوت و چهار پمپ دادهٔ متفاوت برای نمایش حوزه‌های مشتری (`Profile`، `WishList`، `Wallet` و `Preferences`) دارد، اما تنها یک نویسندهٔ داده. همان نویسندهٔ دادهٔ واحدِ مشتری به هر چهار پمپ داده گوش می‌دهد و منطق پایگاه‌داده‌ای (مانند `SQL`) لازم برای به‌روزرسانیِ داده‌های مربوط به مشتری در پایگاه‌داده را در بر دارد.



شکل ۱۶-۱۱. نویسندهٔ دادهٔ حوزه‌محور

راه دیگر این است که هر دسته از واحدهای پردازش مؤلفهٔ `Data Writer` اختصاصیِ خود را داشته باشد؛ چنان‌که در شکل ۱۶-۱۲ نشان داده شده است. در این مدل، هر نویسندهٔ داده به یک پمپ دادهٔ متناظر اختصاص دارد و تنها منطق پردازش پایگاه‌داده‌ایِ همان واحد پردازشِ مشخص (مانند `Wallet`) را در بر دارد. هرچند این مدل معمولاً مؤلفه‌های نویسندهٔ دادهٔ فراوانی پدید می‌آورد، مقیاس‌پذیری و چابکیِ بهتری فراهم می‌کند، زیرا واحد پردازش، پمپ داده و نویسندهٔ داده را با هم هم‌راستا می‌کند.



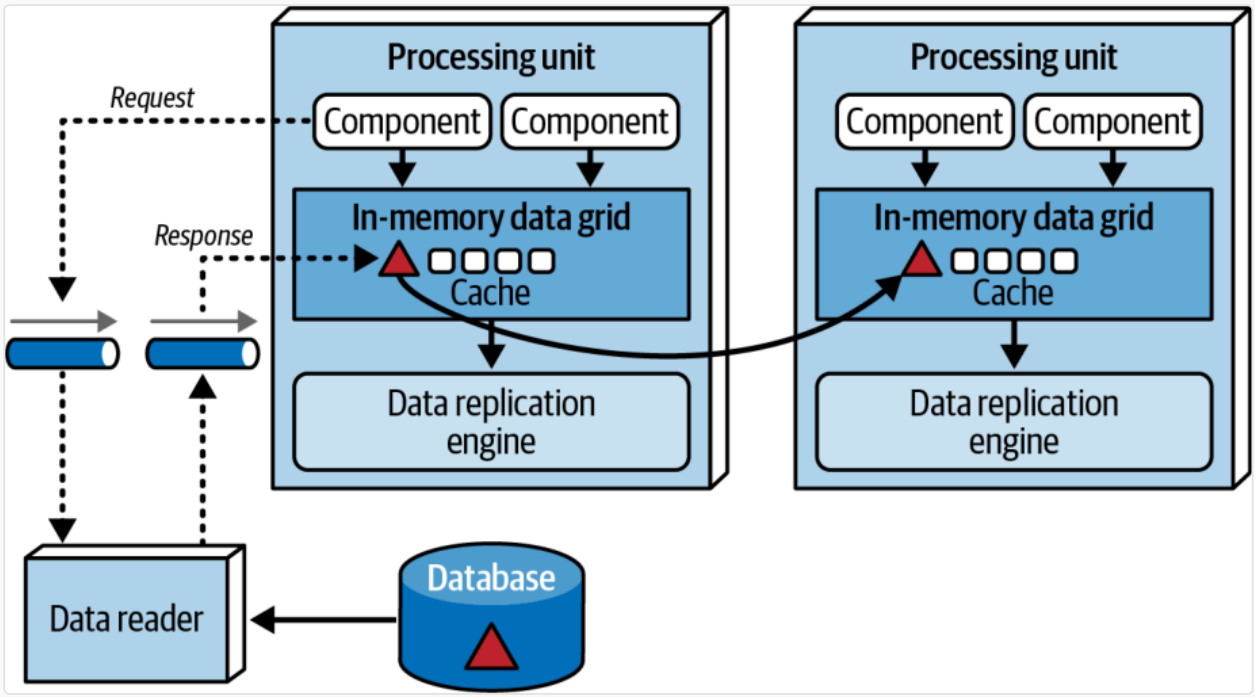
شکل ۱۶-۱۲. نویسنده‌های دادهٔ اختصاصی برای هر پمپ داده

خواننده‌های داده

در حالی که نویسنده‌های داده مسئول به‌روزرسانیِ پایگاه‌داده‌اند، خواننده‌های داده داده‌ها را از پایگاه‌داده می‌خوانند و از راه یک پمپ دادهٔ معکوس `(reverse data pump)` (که کمی بعد بیشتر به آن می‌پردازیم) به واحدهای پردازش می‌فرستند. در معماری‌های مبتنی بر فضا، خواننده‌های داده تنها در یکی از این سه موقعیت فراخوانده می‌شوند: همهٔ نمونه‌های واحد پردازشِ مربوط به یک کشِ هم‌نام از کار بیفتند، همهٔ واحدهای پردازشِ درون یک کشِ هم‌نام دوباره مستقر شوند، یا لازم باشد داده‌های بایگانی‌شده‌ای `(archive data)` که در کش تکثیری وجود ندارند بازیابی شوند.

اگر همهٔ نمونه‌ها به دلیل از کار افتادنِ سراسریِ سیستم یا استقرار دوباره خاموش شوند، داده‌ها باید از پایگاه‌داده در کش بارگذاری شوند؛ کاری که در معماری مبتنی بر فضا معمولاً می‌کوشیم از آن پرهیز کنیم. هنگامی که نمونه‌های یک دسته از واحدهای پردازش دوباره بالا می‌آیند، هر یک می‌کوشد قفلی `(lock)` بر کش به دست آورد. نخستین نمونه‌ای

که قفل را به دست آورد، مالک موقت کش `(temporary cache owner)` می‌شود و بقیه تا آزاد شدنِ قفل به حالت انتظار `(wait state)` می‌روند. (این ممکن است بسته به نوع پیاده‌سازیِ کش متفاوت باشد، اما در هر حال در این سناریو یک مالک اصلی برای کش وجود دارد.) مالک موقت کش برای بارگذاریِ کش پیامی به یک صف می‌فرستد و داده درخواست می‌کند. مؤلفهٔ `Data Reader` درخواست خواندن را می‌پذیرد و سپس منطق پرس‌وجوی پایگاه‌داده‌ایِ لازم برای بازیابیِ داده‌های مورد نیاز را اجرا می‌کند. این مؤلفه داده‌ها را به صف دیگری به نام پمپ دادهٔ معکوس می‌فرستد که آن‌ها را به واحد پردازشی که مالک موقت کش است تحویل می‌دهد. همین که مالک موقت کش را بارگذاری کرد، قفل را آزاد می‌کند. آنگاه همهٔ نمونه‌های دیگر هم‌گام‌سازی می‌شوند و پردازش می‌تواند آغاز شود. این جریان پردازش در شکل ۱۶-۱۳ نشان داده شده است.



شکل ۱۶-۱۳. خواننده‌های داده داده‌ها را به واحدهای پردازش می‌فرستند

خواننده‌های داده نیز، مانند نویسنده‌های داده، می‌توانند حوزه‌محور باشند یا به یک دستهٔ مشخص از واحدهای پردازش اختصاص داشته باشند (حالت دوم رایج‌تر است). پیاده‌سازیِ آن‌ها نیز همانند نویسنده‌های داده است: سرویس، برنامهٔ کاربردی یا هاب داده.

نویسنده‌ها و خواننده‌های داده در اصل یک لایهٔ انتزاعیِ داده `(data abstraction layer)` (یا در برخی موارد، یک لایهٔ دسترسی به داده `(data access layer)`) را شکل می‌دهند. تفاوت این دو در میزان دانش جزئی‌ای است که واحدهای پردازش دربارهٔ طرح‌وارهٔ پایگاه‌داده (ساختار جدول‌ها) دارند. در لایهٔ دسترسی به داده، واحدهای پردازش به ساختارهای دادهٔ زیرینِ پایگاه‌داده کوپل‌اند و تنها به‌طور غیرمستقیم، از راه خواننده‌ها و نویسنده‌های داده، به پایگاه‌داده دسترسی دارند. در مقابل، در لایهٔ انتزاعیِ داده، واحد پردازش با استفاده از قراردادهای جداگانه از طرح‌واره‌های زیرین جدا می‌شود.

معماری مبتنی بر فضا معمولاً به مدل لایهٔ انتزاعیِ داده متکی است تا طرح‌وارهٔ کش تکثیری در هر واحد پردازش بتواند با طرح‌واره‌های زیرینِ پایگاه‌داده متفاوت باشد. این یعنی تغییرات تدریجی (incremental changes) در پایگاه‌داده لزوماً بر واحدهای پردازش اثر نمی‌گذارند. برای تسهیلِ این تغییر تدریجی، نویسنده‌ها و خواننده‌های داده منطق تبدیل (transformation logic) را در بر دارند. اگر نوع یک ستون تغییر کند یا ستون یا جدولی حذف شود، خواننده‌ها و نویسنده‌های داده می‌توانند تغییر پایگاه‌داده را تا زمانی که تغییرات لازم در کش‌های واحد پردازش اعمال شود، حائل کنند.

توپولوژی‌های داده

از آنجا که واحدهای پردازش مستقیماً با پایگاه‌داده تعامل ندارند، معماری مبتنی بر فضا در توپولوژی‌های پایگاه‌داده‌ای که می‌تواند به کار ببرد فوق‌العاده انعطاف‌پذیر است. استفاده از پمپ‌های دادهٔ ناهم‌گام در کنار خواننده‌ها و نویسنده‌های داده یعنی پردازش درخواست (تراکنشی) تا حد زیادی از پایگاه‌داده مستقل است و این امر گزینه‌های گسترده‌ای دربارهٔ توپولوژی و نوع پایگاه‌داده پیش روی معمار می‌گذارد.

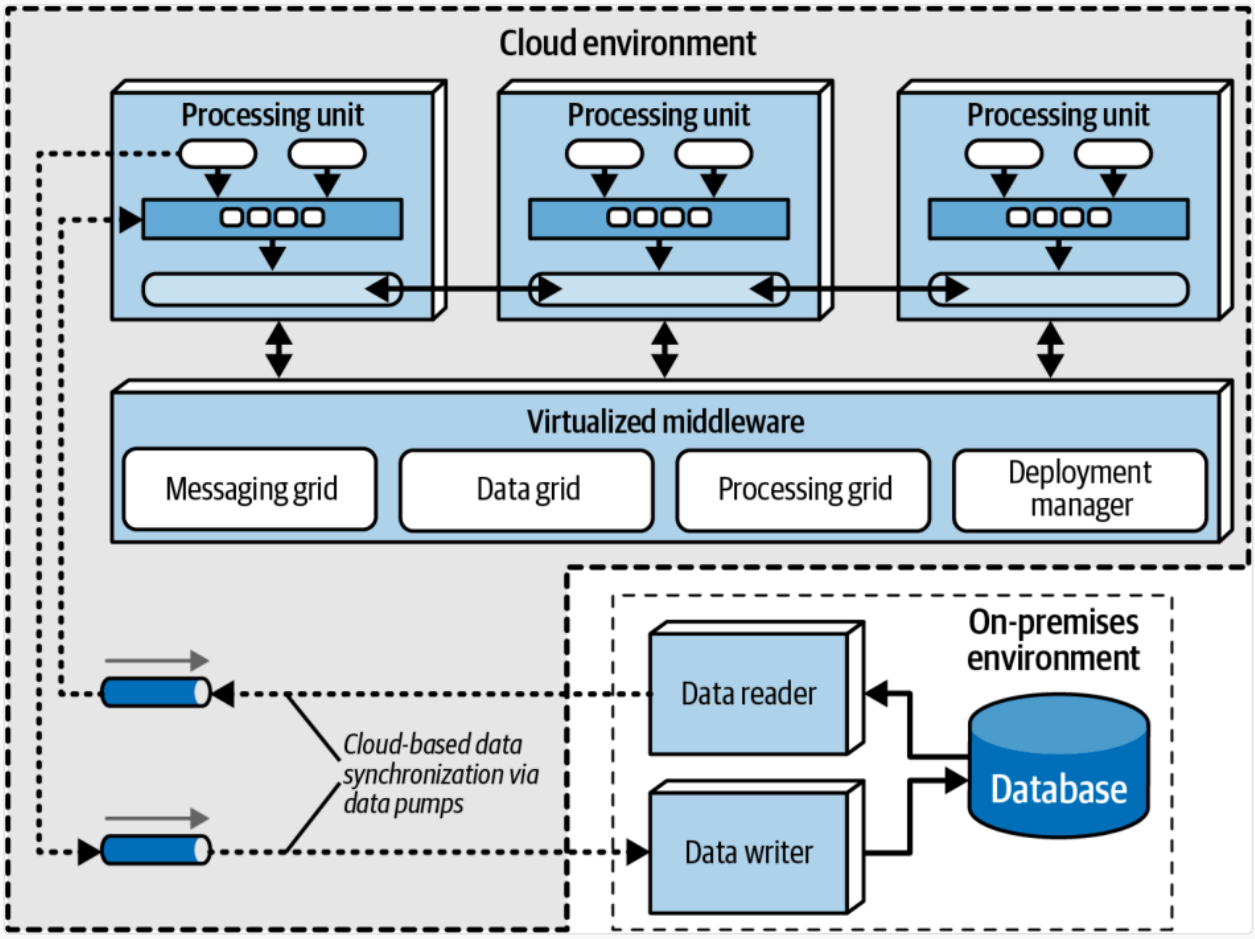
انتخاب توپولوژی پایگاه‌داده تحت تأثیر عوامل فراوانی است. در معماری مبتنی بر فضا، عامل تعیین‌کنندهٔ اصلی این است که سیستم چگونه از پایگاه‌دادهٔ پشتیبان استفاده خواهد کرد. برای نمونه، اگر گزارش‌دهی و تحلیل داده (data analytics) اهمیت ویژه‌ای داشته باشند، توپولوژی پایگاه‌دادهٔ تک‌پارچه ممکن است اثربخش‌تر باشد؛ مگر آنکه گزارش‌دهی و تحلیل داده از راه یک مش داده (data mesh) انجام شود که در این صورت توپولوژی پایگاه‌دادهٔ حوزه‌محور ممکن است بهتر باشد.

توان عملیاتی و سازگاری دادهٔ کلیِ حوزه‌محور نیز در انتخاب توپولوژی پایگاه‌داده از ملاحظات به شمار می‌آیند. یک پایگاه‌دادهٔ تک‌پارچهٔ واحد ممکن است هنگام هم‌گام‌سازی به گلوگاه تبدیل شود، زمان کلیِ هم‌گام‌سازی را کند کند و از سازگاری داده بکاهد؛ اما اگر بتوان داده‌ها را به‌طور تمیز به‌صورت حوزه‌ای افراز کرد، توپولوژی پایگاه‌دادهٔ حوزه‌محور ممکن است زمان کلیِ هم‌گام‌سازیِ بهتر و در نتیجه سازگاری دادهٔ بهتری فراهم کند. سرانجام، اگر سیستم‌های پایین‌دستی برای پردازش‌های بعدی نیاز به استفاده از پایگاه‌داده داشته باشند، توپولوژی پایگاه‌دادهٔ تک‌پارچه ممکن است گزینهٔ مناسب‌تری باشد.

ملاحظات ابری

معماری مبتنی بر فضا در زمینهٔ محیط‌های استقرار گزینه‌های منحصربه‌فردی ارائه می‌دهد. کل سیستم، شامل واحدهای پردازش، میان‌افزار مجازی‌شده، پمپ‌های داده، خواننده‌ها و نویسنده‌های داده و پایگاه‌داده، را می‌توان در محیط‌های مبتنی بر ابر یا به‌صورت درون‌سازمانی (on-prem) مستقر کرد. اما این سبک معماری را می‌توان هم‌زمان در هر دوی این محیط‌ها نیز مستقر کرد (چنان‌که در شکل ۱۶-۱۴ نشان داده شده است)؛ قابلیتی منحصربه‌فرد و قدرتمند که در سبک‌های معماری دیگر یافت نمی‌شود. در این توپولوژی ترکیبی (hybrid topology)، برنامه‌ها از راه واحدهای پردازش و میان‌افزار مجازی‌شده در محیط‌های ابریِ مدیریت‌شده مستقر می‌شوند، در حالی که پایگاه‌داده‌های فیزیکی و

داده‌های متناظرشان درون‌سازمانی نگه داشته می‌شوند. این توپولوژی، به لطف پمپ‌های دادهٔ ناهم‌گام و مدل سازگاری نهاییِ این سبک معماری، از هم‌گام‌سازیِ بسیار اثربخشِ داده‌های مبتنی بر ابر پشتیبانی می‌کند. پردازش تراکنشی می‌تواند در محیط‌های ابریِ پویا و کشسان انجام شود، در حالی که مدیریت فیزیکیِ داده، گزارش‌دهی و تحلیل داده در محیط‌های امنِ درون‌سازمانی باقی می‌مانند.



شکل ۱۶-۱۴. توپولوژی ترکیبیِ مبتنی بر ابر و درون‌سازمانی

ماهیت کشسانِ زیرساخت ابری و سرویس‌های مبتنی بر ابر، چه ترکیبی و چه غیرترکیبی، به‌خوبی با شکل این سبک معماری هم‌خوان است و همین امر محیط‌های مبتنی بر ابر را به گزینهٔ خوبی برای معماری‌های مبتنی بر فضا تبدیل می‌کند.

ریسک‌های رایج

جای شگفتی نیست که بیشتر ریسک‌های مربوط به سبک معماری مبتنی بر فضا، با توجه به استفادهٔ آن از کش‌گذاری و هم‌گام‌سازیِ پس‌زمینه‌ایِ داده، به داده مربوط‌اند. بخش‌های پیش رو برخی ریسک‌های رایج را شرح می‌دهند.

خواندن‌های پرتکرار از پایگاه‌داده

معماری مبتنی بر فضا با استفاده از کش‌گذاری برای همهٔ داده‌های تراکنشی به مقیاس‌پذیری و هم‌روندیِ بالا دست می‌یابد. این کار مانع می‌شود که سیستم خواندن‌ها و نوشتن‌های بیش از اندازه در پایگاه‌داده انجام دهد؛ خواندن‌ها و نوشتن‌هایی که می‌توانند مقیاس‌پذیری، کشسانی و پاسخ‌گوییِ کلیِ سیستم را کاهش دهند و تحقق مزایای این سبک معماری را دشوار کنند.

خواندن از پایگاه‌داده معمولاً تنها در دو سناریو رخ می‌دهد: خواندن داده‌های بایگانی‌شده (مانند تاریخچهٔ سفارش‌ها یا صورت‌حساب‌های بانکیِ گذشته)، یا راه‌اندازی سردِ `(cold-starting)` یک واحد پردازش (نخستین راه‌اندازیِ یک واحد پردازش، هنگامی که هیچ نمونهٔ دیگری در حال اجرا نیست). اگر حجم داده‌های کش‌شده چنان بالا باشد که بیشتر داده‌ها ناچار به بایگانی و بازیابی از پایگاه‌دادهٔ پشتیبان باشند، یا اگر واحدهای پردازش پرتکرار از کار بیفتند یا دوباره مستقر شوند، شاید این معماری برای حوزهٔ مسئله مناسب نباشد.

هم‌گام‌سازی و سازگاری داده

از آنجا که پمپ‌ها و نویسنده‌های داده داده‌ها را میان کش درون‌حافظه‌ای و پایگاه‌داده هم‌گام‌سازی می‌کنند، داده‌ها در معماری مبتنی بر فضا همواره به‌صورت نهایی سازگار خواهند بود. اما از آنجا که این سبک معمولاً در موقعیت‌هایی با بار کاربران هم‌زمانِ بسیار بالا به کار می‌رود، گلوگاه‌ها در پمپ داده رایج‌اند. این گلوگاه‌ها می‌توانند رسیدنِ داده‌ها به پایگاه‌دادهٔ پشتیبان را به‌طور چشمگیری به تأخیر بیندازند. اگر سیستم‌های پایین‌دستی لازم باشد به‌سرعت به داده‌های به‌روز دسترسی داشته باشند، این امر ممکن است ریسک چشمگیری باشد.

ریسک ذاتیِ دیگرِ مربوط به هم‌گام‌سازیِ داده، گم‌شدن داده درون پمپ داده است. این ریسک معمولاً با استفاده از صف‌های ماندگار `(persisted queues)` (که در آن‌ها داده‌های صف هم در حافظه و هم روی دیسک ذخیره می‌شوند) و استفاده از حالت تأیید از سوی کلاینت در نویسنده‌های داده هنگام خواندن از پمپ داده کاهش می‌یابد. حالت تأیید از سوی کلاینت پیام را تا زمانی در صف نگه می‌دارد که نویسندهٔ داده به کارگزار پیام اعلام کند پردازش را به پایان رسانده است. در طول پردازش پیام، کارگزار پیام اطمینان می‌دهد که هیچ نویسندهٔ دادهٔ دیگری نتواند پیامِ در حال پردازش را بخواند. هرچند این تکنیک‌ها به پیشگیری از گم‌شدن داده در پمپ داده کمک می‌کنند، ممکن است پاسخ‌گوییِ کلی را نیز کند کنند و سازگاری داده را کاهش دهند.

حجم بالای داده

از آنجا که همهٔ حافظهٔ تراکنشی در واحدهای پردازش کش می‌شود، حجم داده‌ها باید نسبتاً پایین بماند، به‌ویژه با افزوده‌شدنِ نمونه‌های بیشتری از یک واحد پردازش. از این رو بسیار مهم است که به اندازهٔ کش درون‌حافظه‌ای توجه دقیق شود تا از کمبود حافظه در واحد پردازش و در نتیجه از کار افتادنِ آن پیشگیری شود.

برخوردهای داده

برخورد داده `(data collision)` زمانی رخ می‌دهد که داده‌ای در یک نمونهٔ کش (کش A) به‌روز شود و در جریان تکثیرش به نمونهٔ کشِ دیگری (کش B)، همان داده در آن کش (کش B) نیز به‌روز شود. این وضعیت ممکن است هنگام استفاده از کش‌گذاری تکثیری در حالت فعال/فعال `(active/active)` پیش آید؛ یعنی حالتی که چندین واحد پردازش می‌توانند هم‌زمان یک دادهٔ واحد را به‌روز کنند. برخوردها معمولاً زمانی رخ می‌دهند که نرخ به‌روزرسانیِ داده‌ها در کش از تأخیر تکثیر (replication latency یا `RL`) فراتر رود؛ یعنی از زمانی که `(update rate)` هم‌گام‌سازیِ هر کشِ هم‌نام طول می‌کشد. در این سناریو، به‌روزرسانیِ محلیِ کش B با دادهٔ قدیمیِ کش A بازنویسی می‌شود و همان داده در کش A نیز با به‌روزرسانیِ کش B بازنویسی می‌شود. در نتیجه داده‌های هر کش ناسازگار می‌شوند.

برای نشان‌دادنِ مسئلهٔ برخورد داده، فرض کنید دو نمونه از یک سرویس ثبت سفارش (A و B) وجود دارد که هر یک کشی تکثیری از موجودیِ محصول (ابزارک‌های آبی) دارند. جریان کار به این صورت است:

1. شمار موجودیِ فعلی در هر یک از نمونه‌های A و B برابر با ۵۰۰ واحد است.
2. نمونهٔ A درخواستی از یک مشتری برای خرید ۱۰ واحد دریافت می‌کند و کش موجودیِ ابزارک‌های آبی را به ۴۹۰ واحد به‌روز می‌کند.
3. پیش از آنکه داده‌های نمونهٔ A به نمونهٔ B تکثیر شود، نمونهٔ B درخواست خریدی برای ۵ واحد دریافت می‌کند و کش موجودیِ ابزارک‌های آبی را به ۴۹۵ واحد به‌روز می‌کند.
4. کش نمونهٔ B بر اثر تکثیرِ به‌روزرسانیِ نمونهٔ A به ۴۹۰ واحد به‌روز می‌شود.
5. کش نمونهٔ A بر اثر تکثیرِ به‌روزرسانیِ نمونهٔ B به ۴۹۵ واحد به‌روز می‌شود.
6. هر دو کش نادرست و ناهم‌گام‌اند: موجودی باید در هر دو نمونه ۴۸۵ واحد باشد.

عوامل متعددی بر شمار برخوردهای داده‌ای که ممکن است رخ دهد اثر می‌گذارند: شمار نمونه‌های واحد پردازش که کش یکسانی دارند، نرخ به‌روزرسانیِ کش، اندازهٔ کش و `RL` محصول کش‌گذاری. می‌توانیم بر پایهٔ این عوامل و با استفاده از فرمول زیر، به‌صورت احتمالاتی تعیین کنیم چند برخورد دادهٔ بالقوه ممکن است رخ دهد:

```
\text{CollisionRate} = N \times \frac{UR^2}{S} \times RL
```

در اینجا `$N$` نمایندهٔ شمار نمونه‌های سرویس است که از کش هم‌نام یکسانی استفاده می‌کنند. `$UR$` نمایندهٔ نرخ به‌روزرسانی بر حسب میلی‌ثانیه (به توان دو) است، `$S$` اندازهٔ کش (بر حسب شمار سطرها) است و `$RL$` تأخیر تکثیرِ محصول کش‌گذاری (بر حسب میلی‌ثانیه) است.

این فرمول برای تعیینِ درصد برخوردهای داده‌ای که احتمالاً بر پایهٔ به‌روزرسانی‌ها در یک بازهٔ زمانیِ مشخص (مثلاً هر ساعت) رخ خواهند داد، و در نتیجه برای سنجیدنِ امکان‌پذیریِ استفاده از کش‌گذاری تکثیری در این سیستم، سودمند است. برای نمونه، مقادیر نشان‌داده‌شده در جدول ۱۶-۲ را برای عوامل دخیل در این محاسبه در نظر بگیرید.

جدول ۱۶-۲. مقادیر پایه

عامل	مقدار
نرخ به‌روزرسانی `(UR)` :	۲۰ به‌روزرسانی در ثانیه
شمار نمونه‌ها `(N)` :	۵
اندازهٔ کش `(S)` :	۵۰٬۰۰۰ سطر
تأخیر تکثیر `(RL)` :	۱۰۰ میلی‌ثانیه
به‌روزرسانی‌ها:	۷۲٬۰۰۰ در ساعت
نرخ برخورد `(collision rate)` :	۱۴٫۴ در ساعت
درصد:	۰٫۰۲٪

اعمال این عوامل در فرمول ۷۲٬۰۰۰ به‌روزرسانی در ساعت به دست می‌دهد، با احتمال بالای اینکه ۱۴ به‌روزرسانی روی دادهٔ یکسان با هم برخورد کنند. با توجه به درصد پایین (۰٫۰۲٪)، تکثیر گزینهٔ کارآمدی خواهد بود.

تغییرات `RL` می‌تواند اثر چشمگیری بر سازگاری داده داشته باشد. تأخیر تکثیر به عوامل فراوانی بستگی دارد، از جمله نوع شبکه و فاصلهٔ فیزیکی میان واحدهای پردازش. مقادیر `RL` باید از اندازه‌گیری‌های واقعی در محیط تولید محاسبه و استخراج شوند و به همین دلیل به‌ندرت منتشر می‌شوند. اگر `RL` واقعی در دسترس نباشد، مقدار ۱۰۰ میلی‌ثانیه که در مثال پیشین به کار رفت عدد خوبی برای برنامه‌ریزی است. برای نمونه، تغییر `RL` از ۱۰۰ میلی‌ثانیه به ۱ میلی‌ثانیه همان شمار به‌روزرسانی‌ها (۷۲٬۰۰۰ در ساعت) را به دست می‌دهد، اما احتمال رخدادنِ برخورد را بسیار پایین‌تر می‌آورد (۰٫۱ برخورد در ساعت). این سناریو در جدول ۱۶-۳ نشان داده شده است.

جدول ۱۶-۳. اثر تأخیر تکثیر

عامل	مقدار
نرخ به‌روزرسانی `(UR)` :	۲۰ به‌روزرسانی در ثانیه
شمار نمونه‌ها `(N)` :	۵
اندازهٔ کش `(S)` :	۵۰٬۰۰۰ سطر
تأخیر تکثیر `(RL)` :	۱ میلی‌ثانیه (تغییریافته از ۱۰۰)
به‌روزرسانی‌ها:	۷۲٬۰۰۰ در ساعت
نرخ برخورد:	۰٫۱ در ساعت
درصد:	۰٫۰۰۰۲٪

شمار واحدهای پردازشی که کش هم‌نام یکسانی دارند (که با `$N$` نشان داده می‌شود) نیز با شمار برخوردهای دادهٔ ممکن رابطهٔ مستقیم و متناسب دارد. برای نمونه، کاهش شمار واحدهای پردازش از ۵ نمونه به ۲ نمونه نرخ برخورد داده‌ای برابر با تنها ۶ برخورد در ساعت، از میان ۷۲٬۰۰۰ به‌روزرسانی در ساعت، به دست می‌دهد؛ چنان‌که در جدول ۱۶-۴ نشان داده شده است.

جدول ۱۶-۴. اثر شمار نمونه‌های واحد پردازش

عامل	مقدار
نرخ به‌روزرسانی `(UR)` :	۲۰ به‌روزرسانی در ثانیه
شمار نمونه‌ها `(N)` :	۲ (تغییریافته از ۵)
اندازهٔ کش `(S)` :	۵۰٬۰۰۰ سطر
تأخیر تکثیر `(RL)` :	۱۰۰ میلی‌ثانیه
به‌روزرسانی‌ها:	۷۲٬۰۰۰ در ساعت
نرخ برخورد:	۵٫۸ در ساعت
درصد:	۰٫۰۰۸٪

اندازهٔ کش تنها عاملی است که با نرخ برخورد نسبت معکوس دارد: با کاهش اندازهٔ کش، نرخ برخورد افزایش می‌یابد. در این مثال، کاهش اندازهٔ کش از ۵۰,۰۰۰ سطر به ۱۰,۰۰۰ سطر (با ثابت نگه‌داشتنِ همهٔ عوامل دیگر مانند مثال نخست) نرخ برخوردی برابر با ۷۲ برخورد در ساعت به دست می‌دهد که به‌مراتب بیشتر از حالت ۵۰,۰۰۰ سطر است؛ چنان‌که در جدول ۱۶-۵ نشان داده شده است.

جدول ۱۶-۵. اثر اندازهٔ کش

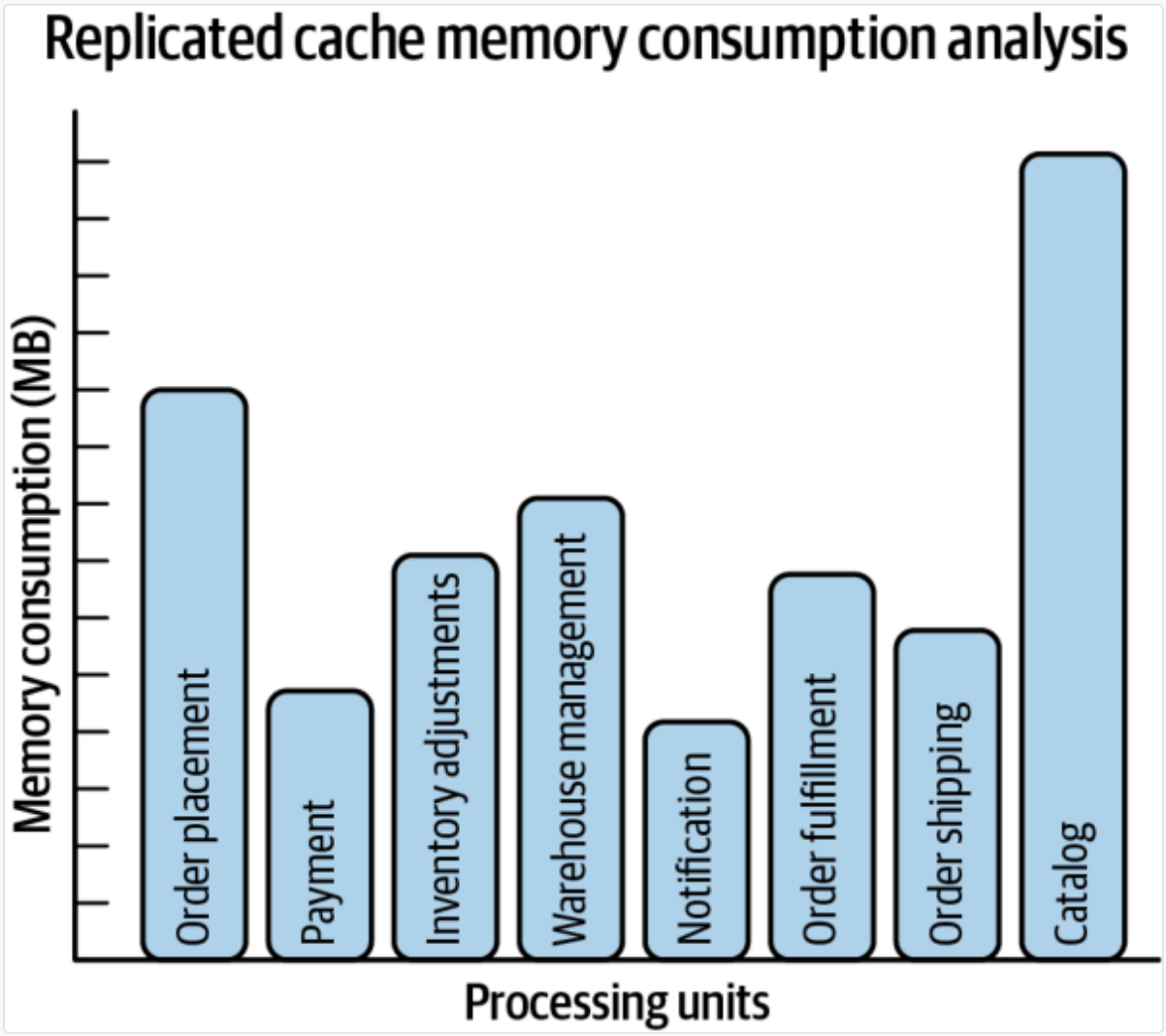
عامل	مقدار
نرخ به‌روزرسانی `(UR)` :	۲۰ به‌روزرسانی در ثانیه
شمار نمونه‌ها `(N)` :	۵
اندازهٔ کش `(S)` :	۱۰,۰۰۰ سطر (تغییریافته از ۵۰,۰۰۰)
تأخیر تکثیر `(RL)` :	۱۰۰ میلی‌ثانیه
به‌روزرسانی‌ها:	۷۲,۰۰۰ در ساعت
نرخ برخورد:	۷۲٫۰ در ساعت
درصد:	۰٫۱٪

در شرایط عادی، نرخ به‌روزرسانیِ بیشتر سیستم‌ها در یک بازهٔ زمانیِ طولانی (مانند یک روز کاریِ هشت‌ساعته) ثابت نیست. هنگام استفاده از این محاسبه، پیشنهاد می‌کنیم بیشینهٔ نرخ به‌روزرسانیِ سیستم خود را در زمان اوج استفاده `(peak usage)` بشناسید و نرخ‌های برخوردِ کمینه، عادی و اوج را محاسبه کنید.

حاکمیت

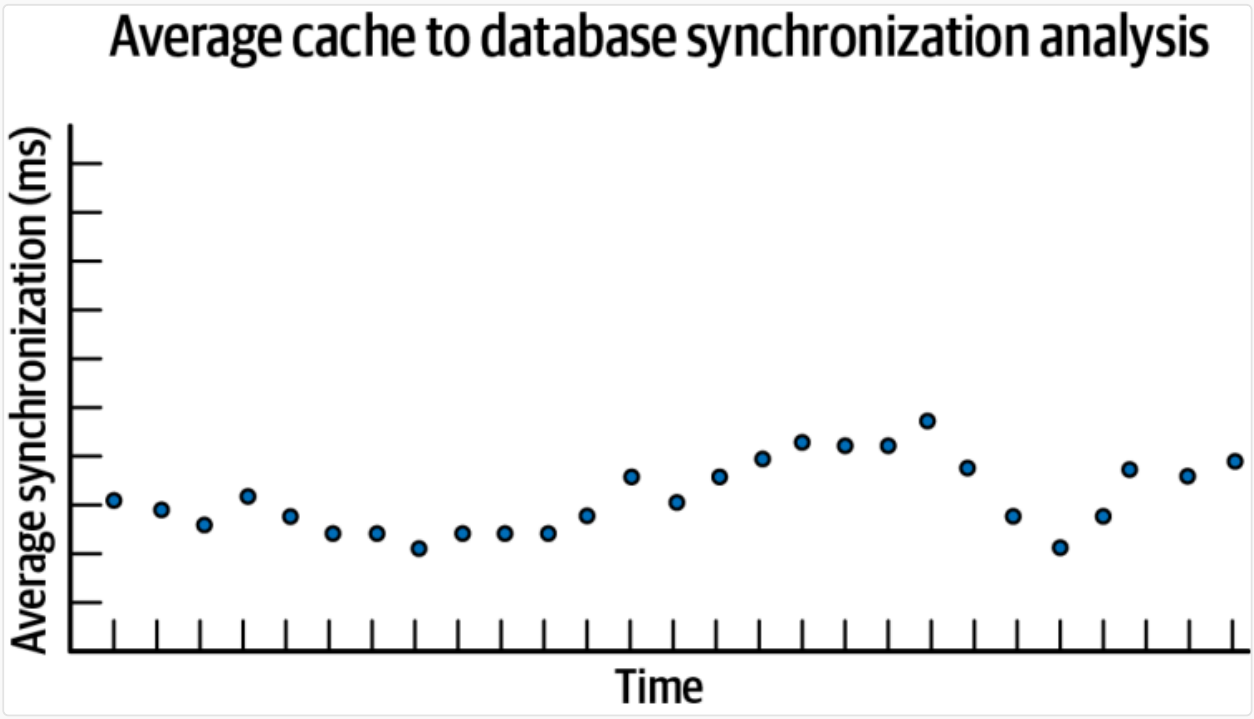
معماری مبتنی بر فضا، با اجزای متحرکِ فراوانش، سبک معماریِ پیچیده‌ای برای طراحی و پیاده‌سازی است. حاکمیت مناسب برای تضمین موفقیت حیاتی است، به‌ویژه برای کنترل مصرف حافظه `(memory consumption)`، با توجه به مسائلی که این سبک در استفاده از حافظهٔ داخلی دارد (چنان‌که پیش‌تر در بخش «ریسک‌های رایج» در همین فصل اشاره کردیم).

برای رسیدگی به این مسائل حافظه، پیشنهاد می‌کنیم یک تابع برازشِ حاکمیتیِ مداوم و خودکار بنویسید تا هر نمونه از یک واحد پردازش، میزان استفادهٔ فعلی‌اش از حافظه را به‌طور دوره‌ای مشاهده‌پذیر کند. از آنجا که همهٔ نمونه‌های یک واحد پردازش کش تکثیریِ یکسانی دارند، تابع برازش تنها باید نام واحد پردازش را گزارش کند. از یک تابع برازش جداگانه برای ثبت شمار نمونه‌های هر واحد پردازش استفاده کنید تا بتوانید مصرف کلیِ حافظه را برای هر واحد پردازش محاسبه کنید. شکل ۱۶-۱۵ نمونه‌ای از خروجیِ این تابع برازش مداوم را نشان می‌دهد.



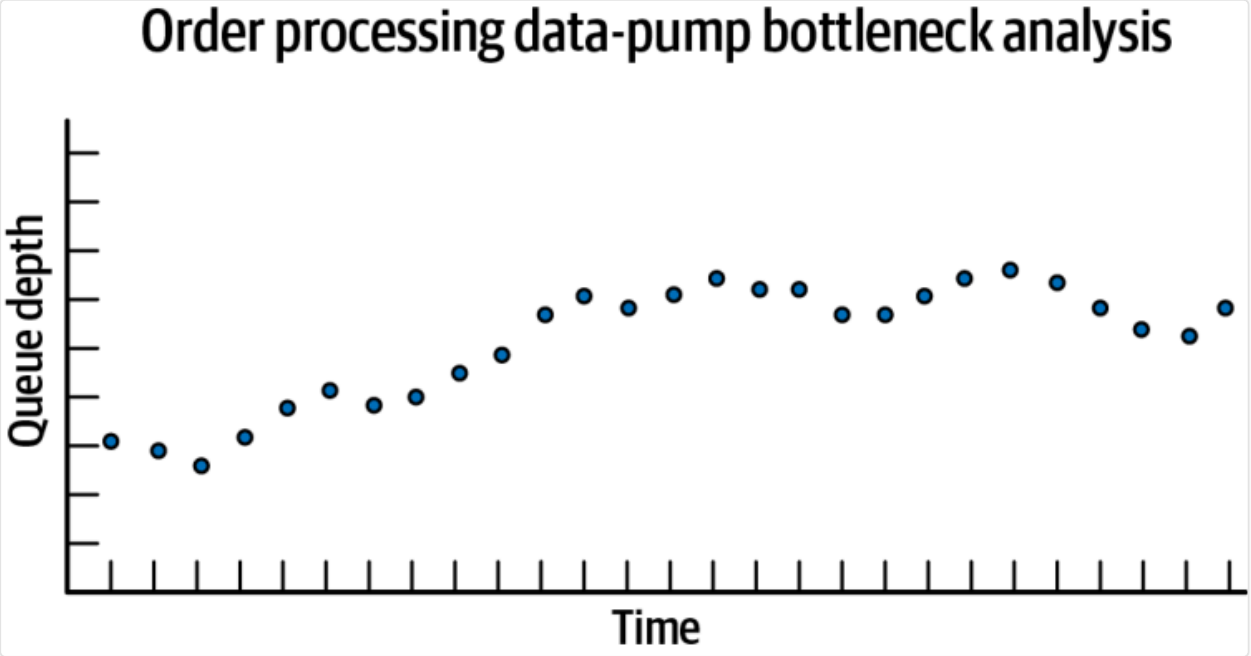
شکل ۱۶-۱۵. نمونه‌ای از تابع برازش برای ردگیریِ مصرف حافظه

راهبرد حاکمیتیِ سودمند دیگر برای حفظ سازگاری دادهٔ کلی، ردگیری و اندازه‌گیریِ زمان هم‌گام‌سازی `(synchronization time)` است؛ به‌طور مشخص، اینکه هم‌گام‌شدنِ یک به‌روزرسانیِ کش با پایگاه‌دادهٔ متناظرش چه مدت طول می‌کشد. یک روش خوب این است که هر واحد پردازش شناسهٔ درخواستِ `(request ID)` یک به‌روزرسانی را همراه با مُهر زمانیِ `(timestamp)` متناظرش به‌صورت جریانی بفرستد و هر نویسندهٔ داده نیز همان شناسهٔ درخواست را همراه با مُهر زمانیِ پس از ثبت در پایگاه‌داده به‌صورت جریانی بفرستد. سپس تابع برازشی بنویسید که این شناسه‌های درخواست را به هم مرتبط کند و با تفریق مُهرهای زمانی، زمان هم‌گام‌سازی را محاسبه کند. تابع برازش می‌تواند این را در سطح اتمی `(atomic)`، با استفاده از زمان‌های مربوط به یک واحد پردازشِ مشخص، یا به‌صورت کل‌نگر `(holistic)`، با میانگین‌گیری از زمان‌های کلیِ هم‌گام‌سازی، ردگیری کند. تحلیل این روندها به معماران کمک می‌کند اثرات تغییرات معماری را ردگیری کنند؛ برای نمونه، اینکه آیا تغییرات زمان‌های هم‌گام‌سازی را بهتر می‌کنند یا بدتر، و آیا معماری به اهداف زمان‌بندیِ هم‌گام‌سازیِ کسب‌وکار دست می‌یابد یا نه. شکل ۱۶-۱۶ این نوع حاکمیت را از راه یک تابع برازش مداوم نشان می‌دهد.



شکل ۱۶-۱۶. نمونه‌ای از تابع برازش برای ردگیریِ میانگینِ تجمیعیِ زمان‌های هم‌گام‌سازی

چنان‌که اشاره کردیم، از آنجا که پمپ‌های داده در معماری مبتنی بر فضا نقش نقطهٔ فشار معکوس را دارند و از آنجا که نوشتن در پایگاه‌داده بیش از نوشتن در کش طول می‌کشد، پمپ‌های داده ممکن است در کل سیستم گلوگاه پدید آورند. برای حاکمیت بر این مسئله، پیشنهاد می‌کنیم این نوع گلوگاه را هنگام پدیدارشدن ردگیری و اندازه‌گیری کنید. نخست، برای تعیین میزان گلوگاه، تابع برازشی بنویسید که عمق صفِ `(queue depth)` صف مورد استفاده در پمپ‌های داده را ردگیری کند. گلوگاهِ بیش از اندازه زمان هم‌گام‌سازی را افزایش می‌دهد (و در نتیجه بر سازگاری داده اثر می‌گذارد) و احتمال گم‌شدن داده و برخوردهای داده را نیز افزایش می‌دهد، به‌ویژه در دوره‌هایی که سطح هم‌زمانیِ کاربران بالاست. این تابع نیز، مانند تابع برازش پیشین، می‌تواند دربارهٔ هر صف پمپ داده به‌صورت اتمی گزارش دهد یا میانگین کلیِ سیستم را تجمیع کند. شکل ۱۶-۱۷ تحلیل گلوگاهی را برای یک واحد پردازش ثبت سفارش و پمپ دادهٔ متناظرش در یک سیستم پردازش سفارشِ متداول نشان می‌دهد.



شکل ۱۶-۱۷. نمونه‌ای از تابع برازش برای ردگیریِ گلوگاه‌های پمپ داده

توابع برازش حاکمیتیِ دیگر برای معماری مبتنی بر فضا می‌توانند بسامد خواندن از پایگاه‌داده (درخواست‌ها به خواننده‌های داده) را ردگیری کنند که می‌تواند بر مقیاس‌پذیری، کشسانی و پاسخ‌گویی اثر بگذارد. همچنین ایدهٔ خوبی است که از توابع برازش برای اندازه‌گیریِ مقیاس‌پذیری، کشسانی و پاسخ‌گویی استفاده کنید: از آنجا که این ویژگی‌های معماری دلایل اصلیِ استفاده از معماری مبتنی بر فضا هستند، ردگیری و اندازه‌گیریِ آن‌ها منطقی است.

ملاحظاتِ توپولوژی تیم

هرچند معماری مبتنی بر فضا، به دلیل مصنوعات فراوانی که هر حوزه یا زیرحوزهٔ مشخص را می‌سازند، تا حد زیادی معماری‌ای افرازشدهٔ فنی به شمار می‌آید، بیشترین اثربخشی را با تیم‌های افرازشده به‌صورت فنی دارد که با حوزه‌های فنیِ آن (مانند کارکرد، پمپ‌های داده، خواننده‌ها و نویسنده‌های داده و مدیریت پایگاه‌دادهٔ پس‌زمینه) هم‌راستا شده‌اند. با این حال، وقتی تیم‌ها بر پایهٔ حوزه سازمان یافته باشند (مانند تیم‌های چندتخصصی با تخصص‌های ویژه) نیز می‌تواند به‌خوبی کار کند. در ادامه نکاتی آمده است که باید دربارهٔ هم‌راستاییِ توپولوژی‌های تیمِ مشخصی که در بخش «توپولوژی‌های تیم و معماری» در فصل ۹ معرفی شدند با معماری مبتنی بر فضا در نظر گرفت:

تیم‌های هم‌راستا با جریان: بسته به اندازهٔ سیستم، تیم‌های هم‌راستا با جریان ممکن است به دلیل افراز فنیِ این سبک معماری، در پیاده‌سازیِ تغییرات حوزه‌محور با دشواری روبه‌رو شوند. برای نمونه، یک تغییر جریان‌محور ممکن است بر یک یا چند واحد پردازش، پمپ داده، خوانندهٔ داده، نویسندهٔ داده، قرارداد کش یا هماهنگ‌کننده، و نیز بر پایگاه‌دادهٔ پشتیبان اثر بگذارد. مدیریت این همه برای یک تیم جریان‌محورِ واحد ممکن است سنگین باشد، به‌ویژه اگر این مصنوعات میان تیم‌های دیگر مشترک باشند. هرچه سیستم بزرگ‌تر و پیچیده‌تر باشد، تیم‌های هم‌راستا با جریان در معماری مبتنی بر فضا کم‌اثرتر خواهند بود.

تیم‌های توانمندساز: از آنجا که برخی مصنوعات این سبک (پمپ‌های داده، خواننده‌های داده، نویسنده‌های داده و میان‌افزار مجازی‌شده) ممکن است مشترک یا میان‌بُرشی باشند، این سبک با تیم‌های توانمندساز تناسب خوبی دارد. اختصاص‌دادنِ یک تیم به یک مصنوعِ مشخص (مانند یک نویسندهٔ داده و پمپ دادهٔ متناظرش) به اعضای آن امکان می‌دهد آزمایش کنند و راه‌هایی برای کاراتر کردنِ این مصنوعات بیابند، مستقل از تیمی که روی کارکرد اصلیِ یک واحد پردازشِ مشخص کار می‌کند.

تیم‌های زیرسیستم پیچیده: تیم‌های زیرسیستم پیچیده می‌توانند از ماهیت افرازشدهٔ فنیِ سبک معماری مبتنی بر فضا بهره بگیرند تا بر بخشی از سیستم (مانند شبکهٔ داده یا پمپ‌های داده) تمرکز کنند. برخی از این مصنوعات ممکن است بسیار پیچیده شوند، از این رو به‌خوبی با توپولوژی تیم زیرسیستم پیچیده سازگارند. رسیدگی به برخوردهای داده (بخش «برخوردهای داده» در همین فصل را ببینید) و دیگر انواع خطاهای هم‌گام‌سازیِ ناهم‌گامِ داده در نویسنده‌های داده کاری بسیار پیچیده است؛ این نمونهٔ بسیار خوبی از زیرسیستم پیچیده‌ای است که تیم‌های حوزه‌محورِ کارکردی که روی واحدهای پردازش کار می‌کنند نباید نگرانش باشند.

تیم‌های پلتفرم: همانند بیشتر سبک‌های معماری، توسعه‌دهندگانی که روی یک معماری مبتنی بر فضا کار می‌کنند می‌توانند با به‌کارگیریِ ابزارها، سرویس‌ها، `API` ها و وظایف مشترک از مزایای توپولوژی تیم پلتفرم بهره ببرند، به‌ویژه اگر بخش‌های زیرساختیِ معماری (مانند پمپ‌های داده و میان‌افزار مجازی‌شده) مرتبط با پلتفرم در نظر گرفته شوند.

ویژگی‌های سبک

در جدول رتبه‌بندیِ ویژگی‌ها در شکل ۱۶-۱۸، رتبهٔ یک‌ستاره یعنی آن ویژگی معماریِ مشخص در این معماری به‌خوبی پشتیبانی نمی‌شود، در حالی که رتبهٔ پنج‌ستاره یعنی آن ویژگی معماری یکی از قوی‌ترین جنبه‌های این سبک معماری است. تعریفِ هر یک از ویژگی‌هایی که در این کارنامه آمده‌اند در فصل ۴ یافت می‌شود.

	Architectural characteristic	Star rating
	Overall cost	$$$$
Structural	Partitioning type	Technical
	Number of quanta	1 to many
	Simplicity	★
	Modularity	★★★
Engineering	Maintainability	★★★
	Testability	★
	Deployability	★★★
	Evolvability	★★★
Operational	Responsiveness	★★★★★
	Scalability	★★★★★
	Elasticity	★★★★★
	Fault tolerance	★★

شکل ۱۶-۱۸. رتبه‌بندیِ ویژگی‌های معماری مبتنی بر فضا

معماری مبتنی بر فضا کشسانی، مقیاس‌پذیری و کارایی را به بیشینه می‌رساند و از همین رو به همهٔ این ویژگی‌ها رتبهٔ پنج‌ستاره داده‌ایم. این‌ها ویژگی‌های پیشران `(driving characteristics)` و مزیت‌های اصلیِ این سبک معماری‌اند. بهره‌گیری از کش‌گذاریِ درون‌حافظه‌ایِ داده و حذف پایگاه‌داده به‌عنوان یک قید، می‌تواند به سیستم‌هایی که با این سبک ساخته می‌شوند سطح بالایی از کشسانی، مقیاس‌پذیری و کارایی ببخشد که برای پردازش میلیون‌ها کاربر هم‌زمان به آن نیاز دارند.

بهای این مزیت‌ها، سادگی و آزمون‌پذیریِ کلیِ سیستم است. معماری‌های مبتنی بر فضا به دلیل استفاده از کش‌گذاری و سازگاری نهاییِ مخزن دادهٔ اصلی (primary data store)، که سیستم مرجعِ نهایی است، بسیار پیچیده‌اند. با توجه به اجزای متحرکِ فراوان این سبک، معماران باید مراقبت ویژه‌ای کنند تا در صورت از کار افتادنِ سیستم، داده‌ای گم نشود (بخش «پیشگیری از گم‌شدن داده» در فصل ۱۵ را ببینید).

آزمون‌پذیری به دلیل پیچیدگیِ شبیه‌سازیِ سطح بالای مقیاس‌پذیری و کشسانی‌ای که این سبک از آن پشتیبانی می‌کند، رتبهٔ یک‌ستاره می‌گیرد. آزمودنِ صدها هزار کاربر هم‌زمان در اوج بار (peak load) کاری بسیار پیچیده و پرهزینه است، از این رو بیشتر آزمون‌های پرحجم در محیط‌های تولید و با بار بسیار سنگینِ واقعی انجام می‌شوند و ریسک چشمگیری برای عملیات عادی به همراه دارند.

هزینه مصالحهٔ دیگری است. معماری مبتنی بر فضا نسبتاً پرهزینه است که بیشتر به دلیل پیچیدگیِ کلی‌اش، هزینه‌های مجوز (licensing fees) محصولات کش‌گذاری و بهره‌برداری از منابعی است که در سیستم‌های ابری و درون‌سازمانی برای پشتیبانی از مقیاس‌پذیری و کشسانیِ بالای آن لازم است.

هرچند واحدهای پردازش، از آنجا که جداگانه مستقر می‌شوند، ممکن است تا اندازه‌ای به افراز حوزه‌ای راه بدهند، ما معماری مبتنی بر فضا را معماری‌ای افرازشدهٔ فنی دانسته‌ایم، زیرا هر حوزهٔ مشخص با مؤلفه‌های فنیِ گوناگونی، مانند واحدهای پردازش، پمپ‌های داده، خواننده‌ها و نویسنده‌های داده و پایگاه‌داده، بازنمایی می‌شود.

تعداد کوانتوم‌ها در یک معماری مبتنی بر فضا، بسته به چگونگیِ طراحی رابط کاربری و شیوهٔ ارتباط واحدهای پردازش، می‌تواند متفاوت باشد. از آنجا که واحدهای پردازش به‌صورت هم‌گام با پایگاه‌داده ارتباط ندارند، خود پایگاه‌داده بخشی از معادلهٔ کوانتوم نیست. در نتیجه، کوانتوم‌ها در یک معماری مبتنی بر فضا معمولاً از راه پیوندهای میان رابط‌های کاربریِ گوناگون و واحدهای پردازش مرزبندی می‌شوند. واحدهای پردازشی که به‌صورت هم‌گام ارتباط برقرار می‌کنند (با یکدیگر یا از راه شبکهٔ پردازش برای ارکستراسیون)، همگی بخشی از یک کوانتوم معماریِ واحد خواهند بود.

نمونه‌ها و موارد کاربرد

معماری مبتنی بر فضا برای برنامه‌هایی که با جهش‌های (spikes) شدید در حجم کاربران یا درخواست‌ها روبه‌رو می‌شوند و برنامه‌هایی که توان عملیاتی‌شان از ۱۰٬۰۰۰ کاربر هم‌زمان فراتر می‌رود، بسیار مناسب است. در اینجا دو مورد کاربرد از معماری مبتنی بر فضا را بررسی می‌کنیم: یک برنامهٔ برخطِ فروش بلیت کنسرت و یک سیستم حراج برخط. هر دو به سطح بالایی از کارایی، مقیاس‌پذیری و کشسانی نیاز دارند.

سیستم فروش بلیت کنسرت

سیستم‌های فروش بلیت کنسرت حوزهٔ مسئلهٔ منحصربه‌فردی‌اند، از این نظر که حجم کاربران هم‌زمان تا پیش از اعلام یک کنسرتِ محبوب نسبتاً پایین است. همین که فروش بلیت‌های یک هنرمندِ به‌ویژه محبوب آغاز شود، حجم کاربران معمولاً، بسته به کنسرت، از چند صد کاربر هم‌زمان به چندین هزار یا حتی ده‌ها هزار جهش می‌کند که همگی می‌کوشند

صندلی‌های خوب به دست آورند! بلیت‌ها معمولاً در عرض چند دقیقه به فروش می‌رسند، از این رو این سیستم‌ها واقعاً به ویژگی‌هایی نیاز دارند که معماری مبتنی بر فضا از آن‌ها پشتیبانی می‌کند.

چالش‌های فراوانی با این نوع سیستم همراه است. نخست، صرف‌نظر از ترجیحات صندلی، تنها شمار مشخصی بلیت در مجموع موجود است. با توجه به شمار بالای درخواست‌های هم‌زمان، وضعیت در دسترس‌بودنِ صندلی‌ها باید پیوسته و با بیشترین سرعتِ ممکن به‌روز شود. دسترسیِ پیوسته و هم‌گام به یک پایگاه‌دادهٔ مرکزی به احتمال زیاد جواب نمی‌دهد؛ برای یک پایگاه‌دادهٔ متداول بسیار دشوار است که ده‌ها هزار درخواست هم‌زمان را از راه تراکنش‌های استاندارد، در این مقیاس و با این بسامد به‌روزرسانی، رسیدگی کند.

معماری مبتنی بر فضا، به دلیل نیازمندی‌های کشسانیِ بالای سیستم فروش بلیت کنسرت، گزینهٔ مناسبی برای آن خواهد بود. مدیر استقرار بی‌درنگ افزایش ناگهانیِ شمار کاربران هم‌زمان را تشخیص می‌دهد و واحدهای پردازشِ فراوانی را برای رسیدگی به حجم بالای درخواست‌های خرید بلیت راه‌اندازی می‌کند. در حالت بهینه، می‌توان مدیر استقرار را چنان پیکربندی کرد که شمار لازمِ واحدهای پردازش را کمی پیش از آغاز فروش بلیت‌ها راه‌اندازی کند تا درست پیش از افزایش چشمگیرِ بار کاربران در حالت آماده‌به‌کار `(standby)` باشند.

سیستم حراج برخط

سیستم‌های حراج برخط (وب‌سایت‌هایی برای پیشنهاد قیمت روی اقلامِ یک حراج، مانند `eBay`) ویژگی‌های مشترک فراوانی با سیستم‌های برخطِ فروش بلیت کنسرت دارند که هم‌اکنون شرح دادیم. هر دو به سطح بالایی از کارایی و کشسانی نیاز دارند و هر دو با جهش‌های پیش‌بینی‌ناپذیر در بار کاربران و درخواست‌ها روبه‌رو هستند. هنگامی که حراجی آغاز می‌شود، هیچ راهی برای تعیین اینکه چند نفر به آن می‌پیوندند یا برای هر قیمت پیشنهادیِ فروشنده `(asking price)` چند پیشنهاد هم‌زمان ثبت خواهد شد، وجود ندارد.

معماری مبتنی بر فضا برای این حوزهٔ مسئله بسیار مناسب است، زیرا امکان می‌دهد با افزایش بار چندین واحد پردازش راه‌اندازی شوند و سپس، هنگامی که حراج رو به پایان می‌رود و دیگر به آن‌ها نیازی نیست، از میان برداشته شوند. می‌توان واحدهای پردازشِ جداگانه‌ای را به هر حراج اختصاص داد تا سازگاری داده‌های پیشنهادها تضمین شود. همچنین، ماهیت ناهم‌گامِ پمپ‌های داده یعنی داده‌های پیشنهادها را می‌توان بدون تأخیر چندانی به پردازش‌های دیگر (مانند تاریخچهٔ پیشنهادها، تحلیل پیشنهادها و ممیزی) فرستاد و بدین ترتیب کارایی کلیِ فرایند پیشنهاددهی را افزایش داد.

معماری مبتنی بر فضا سبک معماریِ پیچیده اما بسیار قدرتمندی است. این تنها سبک معماری است که ترکیب پاسخ‌گویی، مقیاس‌پذیری و کشسانی را به بیشینه می‌رساند؛ که عمدتاً به دلیل شیوهٔ استفاده‌اش از کش‌گذاری و نبودِ دسترسیِ مستقیم به پایگاه‌داده است. از این رو می‌توان آن را سبک معماریِ تخصصی `(specialized architectural style)` دانست که در موقعیت‌هایی به کار می‌رود که باید همین ویژگی‌های معماریِ مشخص را به بیشینه برسانند.

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Space-Based Architecture	معماری مبتنی بر فضا
Request Flow	جریان درخواست
Web Server	سرور وب
Database Server	سرور پایگاه‌داده
Scale Out	مقیاس‌دهیِ افقی
High-Volume Application	برنامهٔ پرحجم
Tuple Space	فضای تاپل
Shared Memory	حافظهٔ مشترک
Replicated In-Memory Data Grid	شبکهٔ دادهٔ درون‌حافظه‌ایِ تکثیرشده
Processing Unit	واحد پردازش
Data Pump	پمپ داده
Persistent Queue	صف ماندگار
Virtualized Middleware	میان‌افزار مجازی‌شده
Messaging Grid	شبکهٔ پیام‌رسانی
Session State	حالت نشست
Data Grid	شبکهٔ داده
Processing Grid	شبکهٔ پردازش
Deployment Manager	مدیر استقرار
Data Writer	نویسندهٔ داده
Data Reader	خوانندهٔ داده
Replication Engine	موتور تکثیر
Load Balancer	توازن‌دهندهٔ بار

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Service Orchestrator	هماهنگ‌کنندهٔ سرویس
Round-Robin Algorithm	الگوریتم نوبت‌گردشی
Next-Available Algorithm	الگوریتم «نخستین واحد در دسترس»
In-Memory Replicated Cache	کش تکثیرشدهٔ درون‌حافظه‌ای
External Controller	کنترل‌گر بیرونی
Remote Call	فراخوانی دور
Caching Provider	ارائه‌دهندهٔ کش
Broadcast	پخش
Member List	فهرست اعضا
Port (network)	درگاه
In-Memory Replicated Caching	کش‌گذاری تکثیریِ درون‌حافظه‌ای
Distributed Caching	کش‌گذاری توزیع‌شده
High Data Volume	حجم بالای داده
Virtual Machine	ماشین مجازی
High-Throughput	پرتوان
Proprietary Protocol	پروتکل اختصاصیِ فروشنده
Mirroring (cache)	آینه‌سازی
Reference Data	دادهٔ مرجع
Data Consistency	سازگاری داده
Single Point of Failure	نقطهٔ شکست واحد
Near-Cache	کش نزدیک
Full Backing Cache	کش پشتیبانِ کامل
Front Cache	کش جلویی

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Eviction Policy	سیاست بیرون‌رانی
Most Recently Used (cache)	تازه‌ترین استفاده
Most Frequently Used (cache)	پرتکرارترین استفاده
Random Replacement	جایگزینیِ تصادفی
Orchestration Engine	موتور ارکستراسیون
Response Time	زمان پاسخ
Message Persistence	ماندگاری پیام
Message Order	ترتیب پیام
Value-Driven Message	پیام ارزش‌محور
Map Message	پیام نگاشتی
Data Hub	هاب داده
Domain-Based Data Writer	نویسندهٔ دادهٔ حوزه‌محور
First-In, First-Out (FIFO)	نخست وارد، نخست خارج
Reverse Data Pump	پمپ دادهٔ معکوس
Archive Data	دادهٔ بایگانی‌شده
Lock	قفل
Temporary Cache Owner	مالک موقت کش
Wait State	حالت انتظار
Data Abstraction Layer	لایهٔ انتزاعیِ داده
Data Access Layer	لایهٔ دسترسی به داده
Incremental Change	تغییر تدریجی
Transformation Logic	منطق تبدیل
Data Analytics	تحلیل داده

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Data Mesh	مش داده
Hybrid Topology	توپولوژی ترکیبی
Cold Start	راه‌اندازی سرد
Persisted Queue	صف ماندگار
On-Premises (On-Prem)	درون‌سازمانی
Client Acknowledge Mode	حالت تأیید از سوی کلاینت
Data Collision	برخورد داده
Active/Active (state)	فعال/فعال
Update Rate	نرخ به‌روزرسانی
Replication Latency (RL)	تأخیر تکثیر
Collision Rate	نرخ برخورد
Peak Usage	اوج استفاده
Memory Consumption	مصرف حافظه
Synchronization Time	زمان هم‌گام‌سازی
Request ID	شناسهٔ درخواست
Timestamp	مُهر زمانی
Atomic (level)	اتمی
Holistic	کل‌نگر
Queue Depth	عمق صف
Continuous Fitness Function	تابع برازش مداوم
Backpressure Point	نقطهٔ فشار معکوس
Driving Architectural Characteristics	ویژگی‌های معماری پیشران
Primary Data Store	مخزن دادهٔ اصلی

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Peak Load	اوج بار
Licensing Fees	هزینه‌های مجوز
Spike (load)	جهش
Standby	آماده‌به‌کار
Asking Price	قیمت پیشنهادیِ فروشنده
Specialized Architectural Style	سبک معماریِ تخصصی
Resource Utilization	بهره‌برداری از منابع
Technically Partitioned Teams	تیم‌های افرازشده به‌صورت فنی